European Journal of Operational Research 291 (2021) 640–660

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/ejor

Discrete Optimization

A matheuristic for parallel machine scheduling with tool replacements

Quang-Vinh Dang*, Thijs van Diessen, Tugce Martagan, Ivo Adan

Department of Industrial Engineering and Innovation Sciences, Eindhoven University of Technology, P.O. Box 513, Eindhoven 5600 MB, the Netherlands

ARTICLE INFO

Article history:
Received 6 November 2019
Accepted 24 September 2020
Available online 6 October 2020

Keywords:
Flexible manufacturing systems
Scheduling
Tool switching
Genetic algorithm
Integer linear programming

ABSTRACT

This paper addresses the problem of scheduling a set of jobs with tool requirements on identical parallel machines in a work center. This problem considers the following characteristics. First, each job may consist of an ordered set of operations due to reentrance to the work center. Moreover, operations have release times and due dates, and the processing of operations requires different tool sets of different sizes. Last, the objective is to minimize both tardiness of operations and tool setup times. Decisions concern the assignment of operations to machines, sequencing of operations, and replacement of tool sets on machines. We propose a mathematical model for the problem and a new matheuristic that combines a genetic algorithm and an integer linear programming formulation to solve industry-size instances. In the matheuristic, we propose two crossover operators which exploit the structure of the problem. We illustrate this approach through real-world case studies. Computational experiments show that our matheuristic outperforms the mathematical model and a practitioner heuristic. We also generate managerial insights by quantifying the potential room for improvement in current practice.

1. Introduction

Global markets have changed towards a customer-centered approach with a growing trend of higher product variety (Rabbani, Montazeri, Farrokhi-Asl, & Rafiei, 2016). Additionally, changes in market demands have been more unpredictable and abrupt over the past decade (Dang, Nguyen, & Rudová, 2019). In order to quickly react on those changes, manufacturers increasingly tend to adopt high-mix low-volume production strategies. In this context, flexible manufacturing systems (FMS) can provide a high level of flexibility in production planning and scheduling with respect to a variety of manufactured products. An FMS mainly consists of computer numerically controlled (CNC) machines able to perform various kinds of operations, storage buffers, and automated material handling devices transporting product components between machines and storage (Shivanand, Benal, & Koti, 2006). Such a system is often employed in metallurgic and microelectronic companies (Paiva & Carvalho, 2017).

In the FMS, each CNC machine is configured with a tool magazine of limited capacity able to hold at least the number of tools needed for processing a single job or operation. A set of needed tools here might be unique for only one operation. Also, the number of tools required for operations has increased due to large product variety in the high-mix low-volume production environment. When the number of required tools exceeds the tool magazine capacity, tool switches, i.e., removing existing tools and then inserting new ones, will become necessary. Even though the tool magazine capacity of CNC machines has increased with advances in new technologies to accommodate more tools, these switches causing tool setup times are still complicating issues. According to Beezão, Cordeau, Laporte, and Yanasse (2017), tool switches consume more time than any other activity in setup operations which accounts for 25–30% of the total cost in an FMS. The switching of tools is hence one of the key factors that influences the FMS efficiency. On the other hand, the ability to deliver products on promised deadlines, especially with "hot" jobs resulting from abrupt demands, is also another key competitive factor since it is reflected in customer satisfaction levels (Weng & Ren, 2006). Therefore, efficient handling of tooling constraints while keeping up with market requirements is crucial for the FMS to improve its utilization and productivity in the high-mix low-volume production environment.

In this paper, we address a problem of parallel machine scheduling with tool replacements in a work center of an FMS. Our problem holds some similarities to the identical parallel machines problem with tooling constraints (IPMTC) addressed by Beezão et al. (2017). The IPMTC considers the processing of a set of given jobs and tool switching on a set of identical parallel machines in order to minimize the makespan, i.e., the time required to complete all jobs. Our problem, however, extends the IPMTC by the

* Corresponding author.
E-mail addresses: q.v.dang@tue.nl (Q.-V. Dang), t.f.h.v.diessen@student.tue.nl (T. van Diessen), t.g.martagan@tue.nl (T. Martagan), i.adan@tue.nl (I. Adan).

https://doi.org/10.1016/j.ejor.2020.09.050

following features to comply with industrial needs. First, the reentrance of jobs to the work center may occur, which means that a job may have two or more ordered operations to be processed. Second, each operation has a release time and a due date. Third, tool sets for processing of operations have non-uniform sizes. The size of a tool set here indicates its number of individual tools. It is notable in our problem that operators switch a whole tool set to another instead of only some individual tools as in the IPMTC. Fourth, multiple criteria, including tool setup time and tardiness of operations, are taken into account. These features, in general, constitute the main novelties of the problem. We aim to make decisions on which machines the operations are assigned, the sequences in which they are processed, and which tool sets have to be removed to insert a required one when there is inadequate magazine capacity, so that the total tardiness of operations and tool setup time is minimized.

Our study in this paper makes the following contributions: (i) we formulate the problem into a mixed-integer linear programming (MILP) model, which is inspired by the work of Beezão et al. (2017), (ii) we propose a new matheuristic approach that outperforms the MILP and a practitioner heuristic, (iii) we provide managerial insights for practitioners by performing sensitivity analyses on industry case studies, which permits us to determine potential room for improvement in current practice, and (iv) we provide data for all real-world case studies used for the experiments in this paper. In our matheuristic which combines a genetic algorithm (GA) and an integer linear programming (ILP) model, we more specifically propose two crossover operators taking advantage of the structure of the problem and a tool replacement method including the ILP formulation. Our computational results reveal that the proposed matheuristic yields on average 20–40% improvement with respect to current practice. The case studies are generated from our industry partner, the *Klein Mechanische Werkplaats Eindhoven* (KMWE) company located in the Netherlands. The core business of the company is to produce CNC-manufactured products that have high complexity and low volumes. The company is hence situated in a high-mix low-volume production environment. Consequently, the company managers can use the proposed solution approach and generated managerial insights in this paper to support their decisions at operational level, such as resource scheduling and allocation, as well as at tactical level, such as capacity planning.

The remainder of the paper is organized as follows. Relevant literature is discussed in Section 2. The problem is formally described in Section 3, and a mathematical model is formulated in Section 4. A matheuristic is developed in Section 5 to solve industry-sized problems, while Section 6 describes a practitioner heuristic which is used at the KMWE company. Next, we present industry case studies and conduct parameter tuning for the proposed matheuristic in Section 7. Computational results are presented in Section 8. Finally, conclusions and future research directions are discussed in Section 9.

2. Literature review

Scheduling of jobs with tool switching in FMS has received much attention in the last decades. Many problems of this type have been considered in the context of single or multiple parallel machines. Problems in the first context are commonly called the job sequencing and tool switching problem (SSP). The SSP consists of two sub-problems, namely the job sequencing problem (JSeP) and tool replacement problem (TRP) (Ahmadi, Goldengorin, Süer, & Mosadegh, 2018). In addition, the SSP is formally proven to be NP-hard for any $C \geq 2$, where C is the tool magazine capacity (Crama, Kolen, Oerlemans, & Spieksma, 1994). On the other hand, problems in the second context can be seen as an extension of the SSP by taking into account multiple parallel machines. This results in an additional sub-problem, that is the parallel machine scheduling problem (PM). Taken together, the PM, JSeP, and TRP form the IPMTC as defined by Beezão et al. (2017). They also conclude that the IPMTC is NP-hard, since the SSP can be considered as its special case. To cope with the SSP and IPMTC as well as their variants, researchers have proposed various exact and heuristic approaches, which are discussed in the following paragraphs.

The SSP with a uniform tool switching time and uniform tool size is introduced and formulated by Tang and Denardo (1988). The problem aims to minimize the number of tool switches. Due to the NP-hard nature of the SSP, exact methods are inherently limited. Tang and Denardo (1988) propose an ILP model for the problem and show that for a given job sequence, the tool switching can be solved in polynomial time by the Keep Tool Needed Soonest (KTNS) policy. This policy states that the tools needed the soonest for upcoming jobs in the sequence should be kept in the magazine when it is necessary to free some slots to accommodate new tools. Laporte, Salazar-González, and Semet (2004) and Ghiani, Grieco, and Guerriero (2010) present different ILP formulations by modeling the SSP as a Traveling Salesman Problem (TSP) and nonlinear least cost Hamiltonian cycle problem respectively. They also propose branch-and-bound (BB) and branch-and-cut (BC) algorithms for solving the SSP. However, their models and approaches are able to solve only small and medium-sized instances, i.e., with up to 25 jobs and 25 tools by the BB of Laporte et al. (2004) and up to 45 jobs and 30 tools by the BC of Ghiani et al. (2010). Karakayalı and Azizoğlu (2006) also develop a BB algorithm and implement several bounding techniques to solve moderate-sized instances of the SSP, but with the objective of minimizing the total flow time. Recently, Catanzaro, Gouveia, and Labbé (2015) present three ILP models for the SSP which are able to produce tighter lower bounds than other alternatives in the literature. However, none of their new formulations can solve instances with more than 10 jobs, 10 tools, and C equal to 6. Schwerdfeger and Boysen (2017) address the problem of sequencing orders from a crane-supplied pick face. Their problem is a variant of the SSP where picking orders substitute jobs, stock keeping units represent tools, and the objective is to minimize the maximum number of switches between successive jobs. They propose an adapted version of the MILP from Tang and Denardo (1988) and a BB procedure, yielding satisfactory results. Furrer and Mütze (2017) study the SSP given multiple objectives being to minimize the number of machine stops (referred to as switching instants) and number of tool switches. An efficient BB algorithmic framework for both real-world and random instances is introduced in their work.

A stream of literature is dedicated to developing heuristics and meta-heuristics approaches for solving the SSP. Several techniques such as job grouping, construction and improvement heuristics can be found in Tang and Denardo (1988), Crama et al. (1994), Hertz, Laporte, Mittaz, and Stecke (1998), Djellab, Djellab, and Gourgand (2000), Salonen, Raduly-Baka, and Nevalainen (2006), Schwerdfeger and Boysen (2017), and Burger, Jacobs, van Vuuren, and Visagie (2015) within the context of the printing industry. On the other hand, meta-heuristic methods based on tabu search (TS) (Al-Fawzan & Al-Sultan, 2003), iterated local search (Paiva & Carvalho, 2017), genetic algorithm (GA) (Ahmadi et al., 2018; Amaya, Cotta, & Fernández, 2008; Chaves, Lorena, Senne, & Resende, 2016), and memetic algorithm (Amaya, Cotta, & Fernández-Leiva, 2011; Amaya, Cotta, & Fernandez-Leiva, 2012; Amaya, Cotta, & Leiva, 2010) have also been developed. Moreover, several authors propose meta-heuristics to solve the SSP with multiple objectives. Keung, Ip, and Lee (2001b) present a GA to simultaneously minimize the total number of tool switches and switch instants for shared tools. The problem of minimizing tool switching and indexing time is addressed by Solimanpur and Rastgordani (2012) with an ant colony

optimization approach. The indexing time is defined as the time within which a magazine can rotate between two neighboring slots or tools. Baykasoğlu and Ozsoydan (2017) and Baykasoğlu and Ozsoydan (2018) solve the same problem for a given sequence of jobs and with tool duplication by developing conventional simulated annealing (SA) and SA with multi starts, respectively. Moreover, some other works consider the SSP with non-uniform tool sizes, where each tool may occupy more than one slot of a magazine. Tzur and Altman (2004) propose a heuristic method called Aladdin and the Keep Smallest Tool Needed Soonest (KSTNS) policy, a variation of the KTNS, to solve the problem of this type. Raduly-Baka, Knuutila, and Nevalainen (2005) present a two-level storage management algorithm for the same problem, which obtains better quality and time performance than the Aladdin heuristic. Van Hop (2005) proposes a construction heuristic to tackle the SSP with unequal tool sizes and relaxed magazine capacity constraint, i.e., a job can require more tools than a magazine can hold. Recently, Calmels (2019) provides a classification scheme for the SSP and its variants and reveals that researchers should focus more on problems with multiple machine systems and multiple objectives, as the problem considered in our paper.

Contrary to the extensively studied SSP, the IPMTC, an extension of the SSP with parallel machines, has not been much investigated in the literature. Several mathematical formulations have been proposed for this type of problems with the objective of minimizing the makespan or number of tool switches. Sarmadi and Gholami (2011) present a mixed integer nonlinear programming (MINLP) model for the IPMTC considering machines with different tool magazine capacities. Their model is capable of solving small problems with up to 15 jobs, 10 tools, and 3 machines only. Ghrayeb, Phojanamongkolkij, and Finch (2003) and Van Hop and Nagarur (2004) present nonlinear programming (NLP) models for the IPMTC, also with different magazine capacities, but in the context of the printed circuit board assembly. Özpeynirci, Gökgür, and Hnich (2016) propose two MILP models to solve the IPMTC considering a limited number of tool copies, but without tool switching times. However, neither model can solve any instances when the number of operations is 15 or more. Recently, Beezão et al. (2017) present two MILP models, inspired by the work of Tang and Denardo (1988) and Laporte et al. (2004), for the IPMTC. Both models also struggle with instances of 15 jobs. For multiple objectives of minimizing the number of tool switches and switching instants, Keung, Ip, and Lee (2001a) formulate an integer programming model for the IPMTC which considers a limited number of tool copies, different magazine capacities, and tool assignments to multiple parallel machines. Their test instance is relatively small with 10 jobs, 9 tools, and 3 machines. Due to the weak performance of the mathematical models, some authors propose heuristic approaches to solve the IPMTC, e.g., construction and improvement procedures (Fathi & Barnette, 2002; Ghrayeb et al., 2003), GA (Keung et al., 2001a; Van Hop & Nagarur, 2004), TS (Özpeynirci et al., 2016), and adaptive large neighborhood search (Beezão et al., 2017). Additionally, a constraint programming approach is considered by Gökgür, Hnich, and Özpeynirci (2018) to obtain better results for the same problem as in Özpeynirci et al. (2016). Lastly, Khan, Gupta, Gupta, and Kumar (2000) present a heuristic procedure to solve the IPMTC in which each job requires a sequence of operations and each operation requires its own set of tools. Nevertheless, their procedure is applicable to only two identical parallel machines and illustrated with a small test instance of 5 jobs and 37 tools.

The problem considered in this paper can be seen as an extension of the IPMTC. It also consists of three interrelated subproblems that must be solved. These include computing the operation assignment to machines, operation sequencing, and tool set allocation on machines, which are represented by the PM, JSeP, and TRP respectively. The problem distinguishes from the IPMTC by the following features: precedence constraints between operations, release times of operations, unequal tool set sizes, and multi-objective minimization of the tardiness and tool setup time. To the best of the authors' knowledge, with this combination of features, the problem has not been addressed in the literature.

A special case of our problem is the SSP, which is known to be NP-hard (Crama et al., 1994). Hence, we conclude that our problem is NP-hard and, as a consequence, intrinsically difficult to solve. To deal with this, we propose a matheuristic approach, which is made by the interoperation of GA and ILP, as presented in Section 5. In our matheuristic, GA is used to solve the two sub-problems PM and JSeP, while ILP is embedded in a KTNS-inspired tool replacement method to cope with TRP. Compared to other optimization methods, GA is known to be effective in performing global search and reducing computational effort. Additionally, GA provides flexibility to hybridize with other approaches to make more efficient implementations for optimization problems (Gen, Cheng, & Lin, 2008). Specifically, our matheuristic integrates ILP into the GA framework as a local search step to enhance the performance of GA. In general, the major advantage of matheuristics is the ability to find the best trade-off between the efficacy of exact approaches and the efficiency of metaheuristics (Guimarães, Klabjan, & Almada-Lobo, 2013). Although matheuristics are a recent trend for solving hard scheduling problems (e.g., Guimarães et al., 2013; Lin & Ying, 2016; Woo & Kim, 2018), to the best of our knowledge, this approach has not been employed yet for the SSP and IPMTC. Results of the matheuristic are compared and evaluated with the help of solutions obtained from a proposed MILP model and a practitioner heuristic as the reference points in the experiments (see Section 8).

3. Problem description

This paper considers a job scheduling and tool switching problem arising in the following flexible manufacturing context. We are given a set of jobs I to be processed by a set of identical parallel machines M, and a set of tool sets T in a work center. Each job $i \in I$ may have to visit the work center more than once before it completes all its processing, which is called reentrance. In such case, each job $i \in I$ consists of an ordered set of n_i operations, where n_i corresponds to the number of times which job i visits the work center. Let $O_i^{\mathcal{I}} = \{(i, j), j = 1 \dots n_i\}$ denote the set of operations of job i and $O = \cup_{i \in I} O_i^{\mathcal{I}}$ denote the set of all operations. For each job i, the order of processing of operations (i, j) given by index j needs to be strictly followed. This can be expressed in the form of precedence constraints $(i, j) \rightarrow (i, j+1)$, where $j = \{1, \dots, n_i - 1\}, n_i > 1$. Each operation (i, j) has a release time r_{ij} and a due date d_{ij}. Each operation must be processed without interruption for a processing time p_{ij} which includes the sequence-independent setup time of that operation.

Each operation (i, j) can be processed by any machine in the work center. Each machine $m \in M$ can perform only one operation $(i, j) \in O$ at a time, and each operation (i, j) can be processed by only one machine at a time. There is sufficient input and output buffer space at each machine. All machines have the same limited capacity C of the magazine to hold tool sets used for processing of operations. Each tool set $t \in T$ has a non-uniform size ϕ_t which indicates the number of individual tools of the tool set t. Each tool set has a sufficient number of copies to meet the needs of production in practice, and thus its limitation is not considered. Each tool set $t \in T$ can be employed for more than one operation. However, each operation requires only one copy of tool set $t \in T$ which is to be loaded into the magazine before processing the operation. The tool set needed for processing of each operation is assigned in advance as a part of the problem definition. Let T_{ij} denote the tool

Table 1
Summary of parameters in Example 1.

Job i	Operation j	Release time r_{ij}	Processing time p_{ij}	Due date d_{ij}	Tool set T_{ij}	Size $\phi_{T_{ij}}$
1	1	0	6	9	5	52
2	1	4	8	13	4	19
3	1	6	5	14	1	21
4	1	6	7	17	4	19
5	1	10	3	15	2	12
1	2	28	5	33	1	21
6	1	25	6	35	7	55
8	1	33	6	39	6	53
7	1	12	4	16	3	19
3	2	20	5	25	2	12

set preassigned for operation (i, j) such that $T = \cup_{(i,j)\in O} T_{ij}$. Note that operations of the same job may require different tool sets.

In most practical situations, a tool magazine with a limited capacity C cannot accommodate all tool sets at once to perform operations, due to the high-mix and low-volume production environment in which our problem is considered. Tool set switches may thus be needed for two successive operations. A switch only occurs when a tool set needed at the start of an operation is not present during processing of the preceding operation on the same machine. In practice, a whole tool set in a magazine is replaced with another one instead of only several individual tools, as assumed in the literature. The main reason is the need for high precision and high quality products. If only part of a tool set in the magazine is replaced, its remaining individual tools have been worn out as compared to newly inserted tools, which may considerably degrade the quality of the products. Therefore, practitioners want to prevent that by changing a whole tool set.

The correct installment of the needed tool set has to be verified after an insertion. The verification is performed by producing one product whose dimensions will be checked by a responsible department. The required time to verify the installment after the insertion of a tool set is called the tool setup time τ. The insertion may require the removal of tool set(s) from a tool magazine to free space. Nevertheless, this removal may in turn cause additional tool setup time if the removed tool set(s) are needed for any succeeding operation on the same machine. Therefore, a smart decision-making mechanism is needed to determine which tool set(s) in the magazine has to be removed for the insertion of the required tool set. This is called the tool replacement subproblem.

The goal is to determine (a) an assignment of operations to machines, (b) a sequence of operations on machines, and (c) a replacement of tool sets on machines such that production constraints are satisfied and performance measures are optimized. The objective is to minimize both total tardiness of operations and total tool setup times of the schedule. Weight coefficients for the total tardiness w_d and tool setup times w_s are introduced to prioritize one measure over the other. Note that a machine stays idle when a new tool set is installed. Hence, by minimizing the tool setup time, we reduce machine idle time, and thus improve machine utilization.

In this paper, the following assumptions are taken into account. First, the size of any tool set is less than or at most equal to the magazine capacity, i.e., $\max\{\phi_t | t \in T\} \leq C$. This assumption is obvious, otherwise, the machine cannot be used due to not-fitted tool sets. Second, for simplicity, we assume that the tool setup time τ is deterministic and constant for all tool sets on all machines. Third, the tool sets initially placed in any tool magazine at the beginning of a schedule do not involve a tool setup time, since they can be verified before the schedule starts. Last, issues such as machine failures or tool breakdowns are not considered.

Example 1. Table 1 below presents a problem of 8 jobs, 2 identical parallel machines, and 7 tool sets. Two of the jobs have reentrant operations, which results in a total of 10 operations. Each machine has a tool magazine capable of storing 80 individual tools. The characteristics of operations and the sizes of tool sets are also presented in Table 1. A feasible schedule for this example is shown in Fig. 1. The upper part of the figure represents the operations on the machines, while the lower part indicates the tools present in the magazines during processing of the corresponding operations. Tool switches are indicated by the dark grey boxes. For instance, at the beginning of the schedule, tool sets 5 and 1 are inserted into machine 2 with the total tool size of 73. They are present on the machine during processing of operations (1,1) and (1,2). However, when machine 2 proceeds to process operation (8,1), tool set 6 of size 53, which is needed for operation (8,1), cannot be inserted in the tool magazine of machine 2 due to its limited capacity. Therefore, a tool switch needs to be performed at the start of operation (8,1), replacing tool set 5 by tool set 6.

4. Mathematical formulation

In this section, we formulate the mathematical model based on the description in Section 3. The model formulation is presented below.

4.1. Decision variables

In addition to the notation presented in Section 3, we introduce the following decision variables for the mathematical formulation.

s_{ij}	starting time of operation (i, j)
e_{ij}	ending time of operation (i, j)
Δd_{ij}	tardiness of operation (i, j)
$x_{iji'j'm}$	equal to 1 if operation (i, j) is followed by operation (i', j') on machine m, 0 otherwise
β_{ijm}	equal to 1 if operation (i, j) is assigned to machine m, 0 otherwise
y_{ijt}	equal to 1 if tool set t is in the magazine during processing of operation (i, j), 0 otherwise
z_{ijt}	equal to 1 if tool set t is inserted at the start of processing of operation (i, j), 0 otherwise

4.2. Mixed-integer linear programming model

The mathematical model of the described problem can be written as follows:

Objective:

$$\min \left(w_d \cdot \sum_{(i,j)\in O} \Delta d_{ij} + w_s \cdot \tau \cdot \sum_{(i,j)\in O} \sum_{t\in T} z_{ijt}\right) \tag{1}$$

Subject to:

$$\Delta d_{ij} \geq e_{ij} - d_{ij} \qquad \forall (i, j) \in O \tag{2}$$

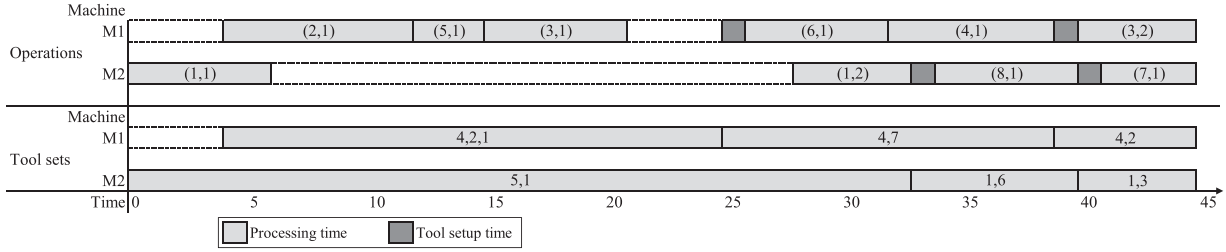

Fig. 1. Gantt chart to illustrate a feasible schedule in Example 1.

$$\Delta d_{ij} \geq 0 \qquad \forall (i,j) \in O \tag{3}$$

Objective (1) minimizes the total weighted tardiness of operations and tool setup time. The tardiness of each operation is defined in Constraints (2) and (3).

$$\sum_{m \in M} \sum_{\substack{(i',j') \in O, \\ (i',j') \neq (i,j)}} x_{iji'j'm} \leq 1 \qquad \forall (i,j) \in O \tag{4}$$

$$\sum_{m \in M} \sum_{\substack{(i,j) \in O, \\ (i,j) \neq (i',j')}} x_{iji'j'm} \leq 1 \qquad \forall (i',j') \in O \tag{5}$$

$$\sum_{m \in M} \beta_{ijm} = 1 \qquad \forall (i,j) \in O \tag{6}$$

$$2 \cdot x_{iji'j'm} \leq \beta_{ijm} + \beta_{i'j'm} \qquad \forall (i',j') \in O, \forall (i,j) \in O \setminus \{(i',j')\}, \forall m \in M \tag{7}$$

$$\sum_{(i,j) \in O} \sum_{\substack{(i',j') \in O, \\ (i',j') \neq (i,j)}} x_{iji'j'm} + 1 \geq \sum_{(i,j) \in O} \beta_{ijm} \qquad \forall m \in M \tag{8}$$

Constraints (4) and (5) make sure that there is at most one predecessor and one successor for each operation, respectively. Constraint (6) ensures that each operation is assigned to only one machine. Constraints (7) and (8) impose the relation between x_{ijm} and β_{ijm} by stating that any two consecutive operations must be performed on the same machine.

$$s_{ij} \geq a_{ij} \qquad \forall (i,j) \in O \tag{9}$$

$$s_{ij} + L(1 - x_{i'j'ijm}) \geq e_{i'j'} \qquad \forall (i,j) \in O, \forall (i',j') \in O \setminus \{(i,j)\}, \forall m \in M \tag{10}$$

$$e_{ij} \geq s_{ij} + p_{ij} + \tau \cdot \sum_{t \in T} z_{ijt} \qquad \forall (i,j) \in O \tag{11}$$

$$e_{ij} \leq s_{i,j+1} \qquad \forall i \in I, j = 1 \ldots n_i - 1, n_i > 1 \tag{12}$$

Constraints (9) and (10) ensure that no operation can start before its release time and before its predecessor on a machine is finished, respectively. We define L in Constraint (10) as a large number, where $L = \max_{(i,j) \in O}\{r_{ij}\} + \sum_{(i,j) \in O} p_{ij} + \tau \cdot |O| \cdot C$ (Beezão et al., 2017; Unlu & Mason, 2010). The completion time of each operation is determined by Constraint (11), and the precedence relationships between operations of the same job are introduced in Constraint (12).

$$\sum_{t \in T} y_{ijt} \cdot \phi_t \leq C \qquad \forall (i,j) \in O \tag{13}$$

$$x_{i'j'ijm} + y_{ijt} - y_{i'j't} \leq z_{ijt} + 1 \qquad \forall (i,j) \in O, \forall (i',j') \in O \setminus \{(i,j)\}, \forall m \in M, \forall t \in T \tag{14}$$

$$y_{ijt} = 1 \qquad \forall (i,j) \in O, \forall t \in T_{ij} \tag{15}$$

$$z_{ijt} = 0 \qquad \forall j \in O, \forall t \in T \setminus \{T_{ij}\} \tag{16}$$

$$x_{iji'j'm} \in \{0,1\} \qquad \forall (i,j), (i',j') \in O, \forall m \in M \tag{17}$$

$$\beta_{ijm} \in \{0,1\} \qquad \forall (i,j) \in O, \forall m \in M \tag{18}$$

$$y_{ijt} \in \{0,1\} \qquad \forall (i,j) \in O, \forall t \in T \tag{19}$$

$$z_{ijt} \in \{0,1\} \qquad \forall (i,j) \in O, \forall t \in T \tag{20}$$

Constraint (13) makes sure that the total size of all tool sets in the magazine does not exceed the capacity during processing of any operation. The need of a tool switch is imposed by Constraint (14), which sets z_{ijt} to one if the tool set needed for the current operation was not present in the magazine during processing of the preceding operation. Constraint (15) enforces the presence of a tool set required by an operation in the magazine while the operation is being processed. Constraint (16) ensures that a tool switch is only performed when the tool set is required to immediately process an operation (Laporte et al., 2004). It implies that tool set t which is not needed for operation (i, j) will not be in the magazine during the operation's processing (i.e., $y_{ijt} = 0$) if the tool set is not inserted at the start of the operation (i.e., $z_{ijt} = 0$) and was not present during the processing of preceding operation (i', j') (i.e., $y_{i'j't} = 0$), according to Constraint (14). The last sets of constraints guarantee valid domains for the remaining variables. We note that $z_{ijt} \in [0, 1]$ is sufficient owing to Constraint (14) and minimization of the tool setup time in Objective (1). Therefore, we can relax Constraint (20) to the following:

$$z_{ijt} \in [0,1] \qquad \forall (i,j) \in O, \forall t \in T \tag{21}$$

Moreover, the existence of identical parallel machines in our problem leads to the existence of many alternative solutions. Given a specific feasible solution, we can obtain an alternative solution by renumbering the machines (Jans, 2009). To exclude those alternatives and thus push the performance of the MILP, we add a symmetry-breaking constraint, which is inspired by Sherali and Smith (2001) as follows.

$$\sum_{(i,j) \in O} \beta_{ij,m-1} \geq \sum_{(i,j) \in O} \beta_{ijm} \qquad \forall m \in M \setminus \{1\} \tag{22}$$

As discussed, our problem is NP-hard and therefore difficult to solve. Moreover, Beezão et al. (2017) demonstrated that their mathematical models for the IPMTC performed well with 8 operations, but struggled for solving when the problem size becomes bigger, even with instances of 15 operations. Our problem is more complex than the IPMTC, which indicates that our MILP model can also be applicable only to small-sized problems in practice. This is demonstrated in the experiments of Section 8, where the MILP model starts to encounter difficulties when increasing the number of operations to 25, even with the added symmetry breaking constraint. The computational time may also grow exponentially in medium and large-sized problems, with more operations or machines. It reinforces the need to develop another method, a matheuristic approach based on the integration of mathematical programming and GA. The matheuristic approach is therefore presented in the next section.

5. Proposed matheuristic

In this section, a matheuristic based on the integration of GA and ILP is developed for solving the problem in Section 3. In our matheuristic, the GA is responsible for determining assignments and sequencing of operations on machines, while the ILP model solves the tool replacement subproblem, i.e., computing which tool set(s) in a machine will be removed to insert a required tool set when needed. The main purpose of integrating the ILP into the GA framework is to utilize the advantages of both GA and ILP to find effective solutions for large problem instances. Let P_k, C_k, and C'_k denote parents, offspring produced by crossover operators, and offspring produced by mutation operators in generation k respectively. In addition, let f_k and f_{best} denote the minimum fitness value of all chromosomes at generation k and the best fitness value over generations, respectively. The pseudocode of the matheuristic is presented in Algorithm 1.

The matheuristic algorithm starts with the initialization of parents P_k where $k = 1$. The algorithm then computes tool replacements for each individual in P_k by an ILP model, whose result is used to calculate the individual's fitness value (lines 3–4). Next, the matheuristic on the one hand will employ two-point and adapted partial-mapped crossovers to create offspring C_k from P_k (line 10). On the other hand, a proposed problem-oriented crossover starts to be used, when a new best solution is found, i.e., $f_k < f_{best}$, until it reaches $\mathcal{B}$ iterations (lines 7–8 and 17–24). The produced offspring C_k are modified to become C'_k by using one-point and swap mutations (line 12). After computing tools replacements and fitness values for all individuals in C'_k, the population for the next generation P_{k+1} is selected from P_k and C'_k based on an elitism selection mechanism (lines 13–15). The matheuristic stops after a predefined maximum computational time *maxTime* or if f_{best} has not improved over G_c consecutive generations. We provide details of particular components in sections corresponding to comments in the pseudocode.

Algorithm 1 Matheuristic.

Require: set of operations O, set of machines M, and set of tool sets T

1: $k \leftarrow 1$, $best \leftarrow$ **false**, $w \leftarrow \infty$
2: Initialize P_k ▷ Section 5.2
3: Compute tool replacements for P_k by ILP ▷ Section 5.6
4: Evaluate P_k ▷ Sections 5.7
5: $f_{best} \leftarrow \min(f_l | \text{chromosome } l \in P_k)$
6: **while** termination criteria not met **do**
7: **if** $best =$ **true** $\vee\ q \leq \mathcal{B}$ **then**
8: Create C_k from P_k by problem-oriented crossover ▷ Section 5.3.2
9: **else**
10: Create C_k from P_k by two-point and adapted partial-mapped crossovers ▷ Section 5.3.1
11: **end if**
12: Create C'_k from C_k by uniform and swap mutations ▷ Section 5.4
13: Compute tool replacements for C'_k by ILP ▷ Sections 5.6
14: Evaluate C'_k ▷ Sections 5.7
15: Create P_{k+1} from P_k and C'_k by elitism selection and immigration ▷ Section 5.5
16: $f_k \leftarrow \min(f_l | \text{chromosome } l \in P_{k+1})$
17: **if** $f_k < f_{best}$ **then**
18: $f_{best} \leftarrow f_k$
19: $best \leftarrow$ **true**
20: $q \leftarrow 1$
21: **else**
22: $best \leftarrow$ **false**
23: $q \leftarrow q + 1$
24: **end if**
25: $k \leftarrow k + 1$
26: **end while**

5.1. Genetic representation

Encoding a solution into a chromosome (or individual) is a key issue for GA. Since the decisions to be made in our problem include the sequencing of operations, it is possibly natural to use a permutation encoding scheme (Gao, Sun, & Gen, 2008). However, the permutation representation can cause infeasible sequences, because there may exist precedence constraints among operations of a job resulting from the reentrance. To cope with this problem, we adopt an encoding scheme proposed by Gao et al. (2008) as follows. An encoded chromosome v consists of a number of n genes, where n is the total number of operations in O such that $n = \sum_{i \in I} n_i$. Each gene v_g $(g = 1, \ldots, n)$ is composed of two parts, namely job i and the machine processing job i. Let $v_g^{\mathcal{I}}$ and $v_g^{\mathcal{M}}$ denote these two parts, thus $v_g = \{v_g^{\mathcal{I}}, v_g^{\mathcal{M}}\}$. Consequently, a chromosome can be represented as $(v_1, \ldots, v_n) = (\{v_1^{\mathcal{I}}, v_1^{\mathcal{M}}\}, \ldots, \{v_n^{\mathcal{I}}, v_n^{\mathcal{M}}\})$. Let $V^{\mathcal{I}} = \{v_g^{\mathcal{I}} | 1 \leq g \leq n\}$ denote the job vector representing the order in which operations of jobs are processed on machines and $V^{\mathcal{M}} = \{v_g^{\mathcal{M}} | 1 \leq g \leq n\}$ denote the machine vector representing the machines selected for the corresponding job $v_g^{\mathcal{I}}$. The job vector $V^{\mathcal{I}}$ here uses the index of a job to represent all operations of that job and interpret them according to the order of occurrences in the sequence. Also, each job i appears exactly n_i times in $V^{\mathcal{I}}$ corresponding to its n_i ordered operations. Consequently, this representation of the job vector $V^{\mathcal{I}}$ always results in a feasible sequence of operations.

Chromosome v (gene v_g)	v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	v_9	v_{10}
Job vector $V^{\mathcal{I}}$	1	2	5	3	6	4	1	8	7	3
Machine vector $V^{\mathcal{M}}$	2	1	1	1	1	1	2	2	2	1
Tool sets	5	4	2	1	7	4	1	6	3	2

Fig. 2. Illustration of solution encoding scheme.

The encoding scheme is illustrated by an example in Fig. 2, which is a solution to the problem of 8 jobs, 10 operations, and 2 machines from Table 1. The order depicted in $V^{\mathcal{I}}$ can be translated into a sequence of operations: (1, 1)≻(2, 1)≻(5, 1)≻(3, 1)≻(6, 1)≻(4, 1)≻(1, 2)≻(8, 1)≻(7, 1)≻(3, 2). Here, we can see that job 1 has two operations represented by $v_1^{\mathcal{I}} = 1$ and $v_7^{\mathcal{I}} = 1$ which are interpreted as the first operation of job 1 (1,1) and the second operation of

job 1 (1,2), respectively. The tool set information displayed at each gene v_g (position g) in Fig. 2 indicates that the tool set required for the corresponding job $v_g^{\mathcal{I}}$ will be present on the corresponding machine $v_g^{\mathcal{M}}$ during processing of $v_g^{\mathcal{I}}$.

5.2. Initialization

The initialization procedure is employed at the first generation or at the generations in which there are duplicated chromosomes as described in Section 5.5. There are in general two methods to generate the initial population, namely heuristic and random initialization. The heuristic initialization may generate individuals having positive fitness values, which can help the GA to find new best solutions faster. On the other hand, the random initialization may explore a large part of the solution space, which facilitates the finding of optimal solutions due to the diversity in the population (Lin, Shinn, Gen, & Hwang, 2006). Hence, we use both initialization methods to create N_p initial chromosomes, where N_p is the population size. Here, one of the individuals is generated by a practitioner heuristic presented in Section 6. For each of the remaining $N_p - 1$ individuals, the job vector $V^{\mathcal{I}}$ is subsequently filled with a random job $i \in I$ from gene v_1 to v_n, subject to the constraint that the occurrence of job i in $V^{\mathcal{I}}$ is equal to its number of operations n_i. Similarly, the machine vector $V^{\mathcal{M}}$ of the chromosome is initialized by subsequently assigning a randomly chosen machine $m \in M$ to $v_g^{\mathcal{M}}$, where $g = 1, \ldots, n$.

5.3. Crossover operators

Crossover is the main genetic operator, which has a great influence on the performance of GA. Crossover operates on two chromosomes at a time to produce an offspring by combining both features of chromosomes (Gen et al., 2008). We here apply the tournament selection method (Gen et al., 2008) to select two parent chromosomes for crossover. In the tournament selection, a number of S_T chromosomes are first randomly chosen from the population where $S_T = \gamma_1 \times N_p$ and γ_1 is a percent of the population size N_p and called tournament selection rate ($0 < \gamma_1 \leq 1$). The chromosome with the best (lowest) fitness value among them is then selected as one parent chromosome. The other parent can also be determined in the same manner. Afterwards, these two parent chromosomes undergo crossover operators, which generates two offspring. The procedure is repeated until there is N_p offspring produced in each generation.

Two crossover operators are proposed in this paper, namely combined crossover (CX) and problem-oriented crossover (POX). The CX itself is composed of two crossover methods, i.e., two-point crossover (2X) and adapted partial-mapped crossover (APMX) while the POX incorporates the Earliest Due Date (EDD) and a constructive heuristic into the crossover. In our matheuristic, the CX is normally employed to create offspring. However, when a new best solution is found, the matheuristic will invoke the POX in turn and use it $\mathcal{B}$ times before switching back to the CX. The main aim of applying the two operators is to take advantages of the more generic crossovers (i.e., 2X and APMX) and the crossover based on heuristics to guide the search more effectively in distinct phases. The two crossover operators, CX and POX, are presented in more detail in the following Sections 5.3.1 and 5.3.2, respectively, while the values of N_p, γ_1, and $\mathcal{B}$ will be discussed in the parameter tuning section.

5.3.1. Combined crossover (CX)

The combined crossover CX consists of a two-point crossover 2X and an adapted partial-mapped crossover APMX which operate on the machine vectors $V^{\mathcal{M}}$ and the job vectors $V^{\mathcal{I}}$ of two parent chromosomes, respectively.

The 2X randomly selects two cutting points and swaps the substrings in between the cutting points of the two parents' machine vectors $V^{\mathcal{M}}$ (Reeves, 2010). The APMX is adapted from a partial-mapped crossover (PMX) proposed by Goldberg and Lingle (1985) which can be considered as an extension of the 2X with additional mapping step to preserve parent information. In the PMX, the value of each gene is unique. Nevertheless, it may not be true for the job vector $V^{\mathcal{I}}$ due to the reentrance of jobs, i.e., a job $v_g^{\mathcal{I}} \in V^{\mathcal{I}}$ may appear more than once. Hence, we propose the APMX method to handle the crossover of $V^{\mathcal{I}}$. The procedure of the APMX is presented below and illustrated by Fig. 3.

- Step 1 Create a transformation scheme and transform values of two parent vectors into unique values according to the transformation scheme.
- Step 2 Select randomly two positions on transformed parent vectors and then exchange the substrings in between these positions to produce proto-offspring vectors.
- Step 3 Determine the mapping relationship between the substrings and then legalize proto-offspring vectors by changing values outside of the substrings according to the mapping relationship.
- Step 4 Convert values of proto-offspring vectors back into the job indexes to obtain offspring vectors based on the transformation scheme.

Compared to the PMX, the proposed APMX adds two additional steps, i.e., steps 1 and 4, which work according to a transformation scheme. The transformation scheme is created by mapping each job $v_g^{\mathcal{I}} \in V^{\mathcal{I}}$ of a parent, i.e., parent 1, onto its position g ($1 \leftrightarrow 1, \ldots, 3 \leftrightarrow 10$). Subsequently, both parent vectors are transformed based on this scheme as shown in step 1 of Fig. 3. Two proto-offspring vectors PO1 and PO2 produced in step 2 may contain duplicated values and may not have some values, e.g., 1, 9, and 10 are duplicated in the PO1 while 4, 5, and 6 are missing from it. Then, according to the mapping relationship between the substrings determined in step 3, the duplicated values 1, 9, and 10 in the PO1 should be replaced by the missing values 4, 5, and 6, respectively while keeping the exchanged substring unchanged (legalization). Finally, the PO1 and PO2 are converted into the offspring vectors O1 and O2, respectively according to the transformation scheme as illustrated in step 4 of Fig. 3. Note that all vectors mentioned in the APMX are job vectors.

5.3.2. Proposed problem-oriented crossover (POX)

The presented 2X and APMX of the CX can be viewed as generic crossovers, which do not take into account problem characteristics while producing offspring. Although these generic crossovers are helpful to diversify the search, they may also waste computational resources in exploring unpromising regions, which results in slower convergence. Thus, in addition to the CX, we propose a problem-oriented crossover POX considering problem characteristics, i.e., due date and tool switches, which may help not only to guide the search for more promising areas, but also tune the search towards the best solutions faster. The proposed POX consists of two methods, each of which creates each vector of an offspring as follows.

First, to produce the job vector of an offspring, the POX extends the APMX procedure by adding one additional step in which the EDD rule is incorporated. This additional step will rearrange the jobs outside of the exchanged substring in non-decreasing due date order of their corresponding operations. By doing so, the POX aims to reduce the tardiness of a schedule. Fig. 4 below illustrates the additional step where offspring (job) vector O1 is changed accordingly to obtain revised offspring (job) vector RO1. The other revised offspring vector RO2 can be produced in the same manner.

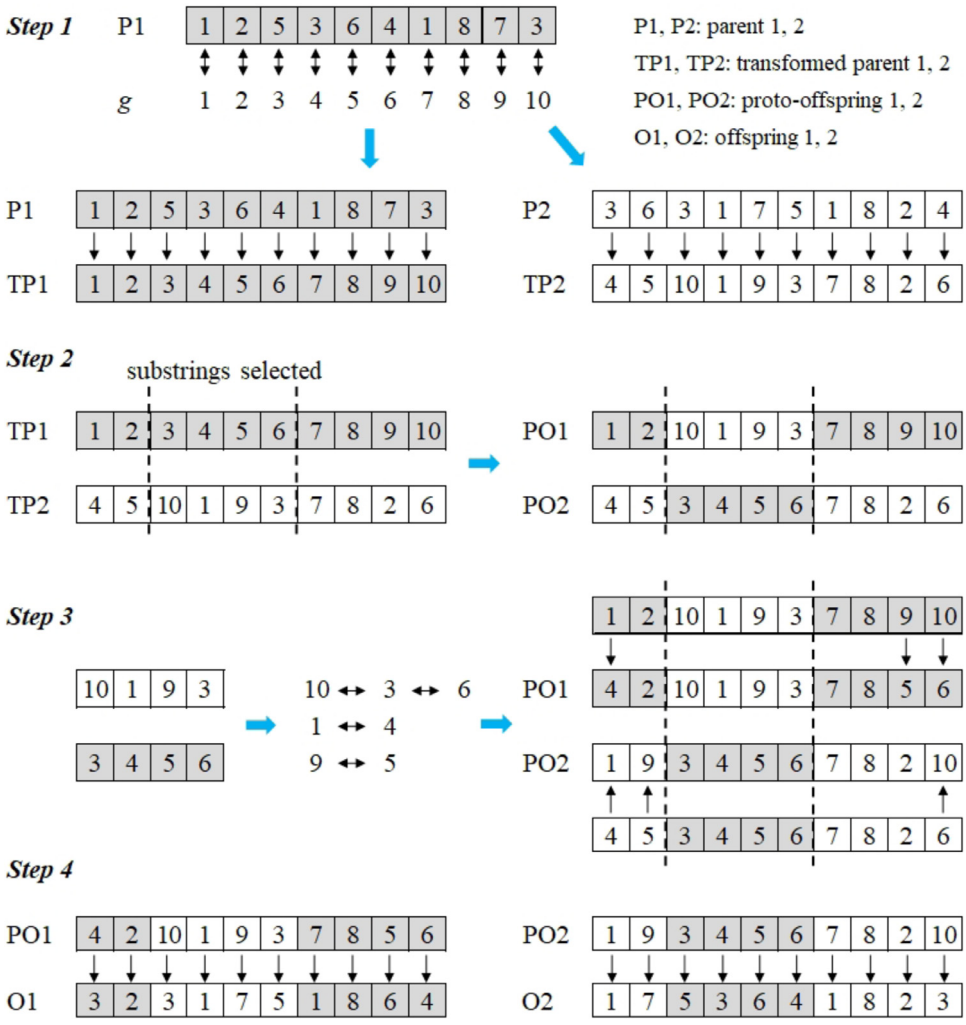

Fig. 3. Illustration of the APMX procedure.

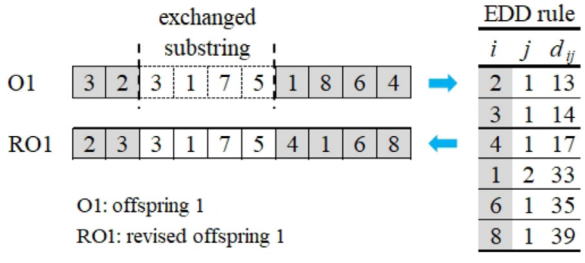

Fig. 4. Illustration of incorporating EDD rule into APMX.

Second, by using the produced job vector as input, the POX builds up the machine vector by allocating a machine to each corresponding job successively from gene v_1 to v_n based on a constructive heuristic. Let $T_m^{\mathcal{M}}$ denote the set of tool sets present on machine m, m_T denote a machine holding the tool set T_{ij} required for a considered operation (i, j), and $M_C \subseteq M$ denote a subset of machines in which each has enough remaining capacity to insert T_{ij}, so $M_C = \{m \in M | C - \sum_{t \in T_m^{\mathcal{M}}} \phi_t \geq \phi_{T_{ij}}\}$. Also, let $p_m^{\mathcal{M}}$ denote the sum of processing times of the operations already allocated to machine $m \in M$. The pseudocode of the constructive heuristic is presented and explained below.

The constructive heuristic first interprets job $v_g^{\mathcal{I}}$ ($v_g^{\mathcal{I}} = i | i \in I$) in an input job vector to determine its corresponding operation (i, j) and defines a machine m_T which holds tool set T_{ij} required for this operation (lines 4–5). If there exists such machine m_T, it is selected (line 6–7). If the required tool set T_{ij} is not present in any machine, the heuristic will take into account machines in the subset M_C which have sufficient remaining capacity to insert tool set T_{ij}. Then, if multiple machines exist in M_C, the one with the smallest $p_m^{\mathcal{M}}$ is chosen and tool set T_{ij} is inserted into the chosen machine (lines 9–12). On the other hand, if there is no machine with sufficient remaining capacity, i.e., $M_C = \emptyset$, then a machine in the set of all machines M which has the smallest $p_m^{\mathcal{M}}$ is selected (lines 13–14). Finally, the total processing time of scheduled operations on the chosen machine m^* is updated and machine m^* is recorded as $v_g^{\mathcal{M}}$ in $V^{\mathcal{M}}$ (lines 17–18). The constructive heuristic continues until all values in $V^{\mathcal{M}}$ are filled. One can note that allocating operation (i, j) to a machine which already holds its required tool T_{ij} aims to reduce the tool setup time. In addition, the heuristic has a tendency to balance the total processing times $p_m^{\mathcal{M}}$ of machines in hopes of reducing the total tardiness.

Example 2. The constructive heuristic is illustrated by building up a machine vector of an offspring with the job vector RO1 from Fig. 4 and other information from Table 1 (also Example 1) as input. We illustrate three cases in Algorithm 2 corresponding to the solution in Fig. 5 as follows.

- $g = 1$ (Case 2): operation (2,1) requires tool set 4 with the size of 19. At the beginning, the tool magazines of all machines are

Algorithm 2 Constructive heuristic.

Require: Offspring's job vector, required tool sets of jobs, processing times of operations

$T_m^{\mathcal{M}} \leftarrow \emptyset,\ \forall\ m \in M$
$p_m^{\mathcal{M}} \leftarrow 0,\ \forall\ m \in M$
for $g = 1$ to n **do**
  $(i, j) \leftarrow \text{interpret}(v_g^{\mathcal{I}})$
  $m_T \leftarrow \{m \in M | T_{ij} \in T_m^{\mathcal{M}}\}$
  if $\exists m_T$ **then**
    $m^* \leftarrow_T$ ▷ Case 1
  else
    $M_C \leftarrow \{m \in M | C - \sum_{t \in T_m^{\mathcal{M}}} \phi_t \geq \phi_{T_{ij}}\}$
    if $|M_C| \geq 1$ **then**
      $m^* \leftarrow \text{argmin}(p_m^{\mathcal{M}} | m \in M_C)$ ▷ Case 2
      $T_{m^*}^{\mathcal{M}} \leftarrow T_{m^*}^{\mathcal{M}} \cup T_{ij}$
    else
      $m^* \leftarrow \text{argmin}(p_m^{\mathcal{M}} | m \in M)$ ▷ Case 3
    end if
  end if
  $p_{m^*}^{\mathcal{M}} \leftarrow p_{m^*}^{\mathcal{M}} + p_{ij}$
  $v_g^{\mathcal{M}} \leftarrow m^*$
end for

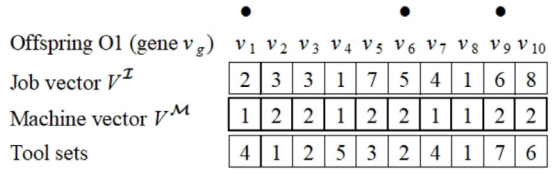

Fig. 5. Machine vector of offspring produced by the POX.

empty, which means that tool set 4 is not present on any machine and it can be placed into one of the machines in the set $M_C = \{1, 2\}$. In addition, no operations have been scheduled at this stage, thus $p_1^{\mathcal{M}} = p_2^{\mathcal{M}} = 0$. Consequently, operation (2,1) with tool set 4 can be allocated to any machine in the M_c. Here, we break the tie by selecting the machine with the lowest index, so $v_1^{\mathcal{M}} = m^* = 1$, $p_1^{\mathcal{M}} = 8$ and $T_1^{\mathcal{M}} = \{4\}$.

- $g = 6$ (Case 1): operation (5,1) requires tool set 2 with the size of 12. Given that $T_1^{\mathcal{M}} = \{4, 5\}$ and $T_2^{\mathcal{M}} = \{1, 2, 3\}$ due to the placement of tool sets of the preceding operations. It can be seen that machine 2 already holds tool set 2, so $M_T = \{2\}$. Consequently, $v_6^{\mathcal{M}} = m^* = 2$ and $p_2^{\mathcal{M}} = 17$.
- $g = 9$ (Case 3): operation (6,1) requires tool set 7 with the size of 55. Given $T_1^{\mathcal{M}} = \{4, 5\}$ and $T_2^{\mathcal{M}} = \{1, 2, 3\}$, there is no machine which holds tool set 7 in their magazines. In addition, the remaining capacity of machines 1 and 2 are respectively 9 and 28, so $M_C = \emptyset$. Here to select a machine, the total processing times of all machines are considered ($p_1^{\mathcal{M}} = 26$ and $p_2^{\mathcal{M}} = 17$) and the machine with the smallest $p_m^{\mathcal{M}}$ is chosen, so $v_9^{\mathcal{M}} = m^* = 2$ and $p_2^{\mathcal{M}} = 23$.

5.4. Mutation

Mutation is a genetic operator which produces spontaneous random changes in various chromosomes (Gen et al., 2008). In our matheuristic, we apply two mutation operators, namely swap and uniform mutations that operate on the job and machine vectors of every produced offspring, respectively.

In the swap mutation, each gene v_g has a probability P_s to be selected for mutation. If so, then $v_g^{\mathcal{I}}$ of the selected gene v_g is swapped with $v_{g'}^{\mathcal{I}}$ of another gene $v_{g'}$ which is randomly chosen. For the uniform mutation, each gene v_g will be now selected with a probability P_u. Then, the operator replaces $v_g^{\mathcal{M}}$ of gene v_g with a discrete uniform value randomly chosen in the range [1, $|M|$]. The mutation probabilities P_s and P_u are determined in the parameter tuning section. The two mutation operators are illustrated by Fig. 6.

5.5. Elitism selection and immigration

The driving force in GA is the selection of individuals based on their fitness to generate a population of the next generation (Du & Swamy, 2016). In this paper, an elitism selection operator is proposed. The elitism selection ranks all parent chromosomes in non-decreasing fitness values and then selects the first fraction γ_2 of the ranked parents. In other words, the operator selects a number of S_E best parents with the lowest fitness where $S_E = \gamma_2 \times N_p$ and γ_2 is defined as elitism selection rate ($0 < \gamma_2 \leq 1$). Next, it randomly selects S_E offspring chromosomes and replaces them with the S_E best parents chosen previously. These S_E best parents together with the remaining offspring form the population of the next generation.

The population of the next generation is finalized by replacing all duplicated chromosomes with new chromosomes which are randomly generated according to the initialization procedure described in Section 5.2 (this step is called immigration). The process of immigration and elitism selection allows increased exploration while maintaining nearly the same level of exploitation for the given population size (Gen et al., 2008).

5.6. Proposed tool replacement method

The genetic operators of GA described thus far are used to solve the assignment of operations to machines and sequencing of operations on machines. In this section, we propose a method including an ILP model to solve the tool replacement subproblem that determines which tool set(s) to be removed from a tool magazine for the insertion of a required tool set. The proposed tool replacement method (TRM) is inspired by the KTNS policy introduced by Tang and Denardo (1988). The TRM is employed only if the removal of any tool set is inevitable, which means, when there is not enough remaining capacity to insert a required tool set into the magazine. The TRM consists of three steps as follows: (i) determine which tool set(s) are still needed for succeeding operations on a machine, (ii) assign a score to each tool set based on the order at which it is needed, and (iii) compute which tool set(s) are removed from a tool magazine based on the ILP considering given scores. Each of the steps in the TRM is explained in more detail below.

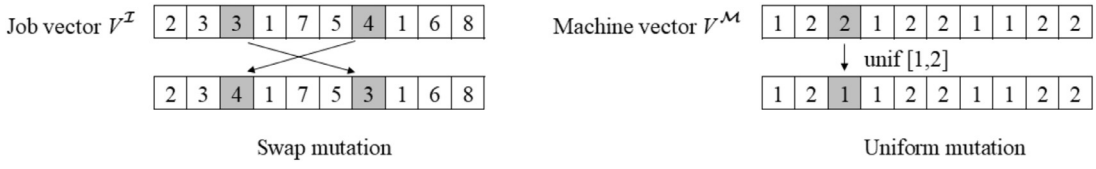

Fig. 6. Mutation operators.

Table 2
Operations scheduled on machine 1.

i	j	T_{ij}	$\phi_{T_{ij}}$
2	1	4	19
5	1	2	12
3	1	1	21
6	1	7	55
4	1	4	19
3	2	2	12

In step (i), let $TS_m^{\mathcal{M}} \subseteq T_m^{\mathcal{M}}$ denote a subset of the set of all tool sets in machine m which will be used later for any succeeding operations scheduled on that machine. Then, the TRM checks and determines $TS_m^{\mathcal{M}}$ for each machine m.

Example 3. We consider the chromosome from Fig. 2 in which machine 1 in the machine vector is investigated. The sequence of operations on this machine can be interpreted as: (2, 1)≻(5, 1)≻(3, 1)≻(6, 1)≻(4, 1)≻(3, 2). The information related to these operations is shown in Table 2 (which is extracted from Table 1). Initially, machine 1 with the magazine capacity of 80 can hold tool sets 4, 2, and 1 that are required for processing of three operations (2,1), (5,1), and (3,1), respectively. Thus, $T_1^{\mathcal{M}} = \{4, 2, 1\}$ with a total size of 52 and 28 free places left in the magazine. Since tool set 7 of size 55 is essential to process operation (6,1), the removal of tool sets is unavoidable due to insufficient remaining capacity. Then, the TRM determines that tool sets 4 and 2 are still needed for the production of succeeding operations (4,1) and (3,2) respectively, so $TS_1^{\mathcal{M}} = \{4, 2\}$.

Next, in step (ii), each tool set $t \in TS_m^{\mathcal{M}}$ is given a score sc_t according to the order which it is needed in the sequence $O^{\mathcal{S}}$ of succeeding operations. One can note that each tool set may be required by more than one operation in the $O^{\mathcal{S}}$. Hence, we take into account a subsequence $OU^{\mathcal{S}} \subseteq O^{\mathcal{S}}$ containing only the operation that first requires each tool set $t \in TS_m^{\mathcal{M}}$ in the $O^{\mathcal{S}}$. Then, the score $sc_t = |TS_m^{\mathcal{M}}| - (u-1)$ is assigned to each tool set $t \in TS_m^{\mathcal{M}}$ which the uth operation in the $OU^{\mathcal{S}}$ requires for processing. It can be seen that the highest score $|TS_m^{\mathcal{M}}|$ is given to a tool set required by the first operation in the $OU^{\mathcal{S}}$ and similarly the score of $|TS_m^{\mathcal{M}}| - 1$ is for a tool set required by the second operation in the $OU^{\mathcal{S}}$, etc. Any remaining tool sets in $T_m^{\mathcal{M}}$ which are not needed anymore (i.e., $T_m^{\mathcal{M}} \setminus TS_m^{\mathcal{M}}$) is given a score of 0.

Example 4. In Example 3, we have $TS_1^{\mathcal{M}} = \{4, 2\}$. Scores are assigned to tool sets 4 and 2 in $TS_1^{\mathcal{M}}$ based on the order they are required in the sequence of succeeding operations $O^{\mathcal{S}} = \{(6, 1), (4, 1), (3, 2)\}$. Here, tool sets 4 and 2 are first employed by operations (4,1) and (3,2) in the $O^{\mathcal{S}}$ respectively, hence $OU^{\mathcal{S}} = \{(4, 1), (3, 2)\}$. Then, since (4,1) is the first operation in the $OU^{\mathcal{S}}$ requiring a tool set in $TS_1^{\mathcal{M}}$, i.e., tool set 4, we have $sc_4 = 2 - (1 - 1) = 2$. Subsequently, $sc_2 = 2 - (2 - 1) = 1$. Tool set 1 in $T_1^{\mathcal{M}}$ is not present in $TS_1^{\mathcal{M}}$ because it is not needed for any succeeding operations, thus $sc_1 = 0$.

Finally, in step (iii), the decision as to which tool sets should be removed from $T_m^{\mathcal{M}}$ has to be made considering given scores in step (ii) to obtain at least sufficient capacity $\phi_m^{\mathcal{S}}$ for inserting the required tool set T_{ij}. We define the sufficient capacity $\phi_m^{\mathcal{S}}$ on machine m as the additional space that should be freed apart from the remaining capacity to insert T_{ij}, so $\phi_m^{\mathcal{S}} = \phi_{T_{ij}} - (C - \sum_{t \in T_m^{\mathcal{M}}} \phi_t)$.

If the total size of the tool sets in $T_m^{\mathcal{M}}$ that have a score of 0 is greater than or equal to $\phi_m^{\mathcal{S}}$, then these tool sets are removed and an ILP model proposed later in step (iii) does not need to be used. Otherwise, we consider the combinations of tool sets in $T_m^{\mathcal{M}}$ to be possibly removed. The number of combinations is $2^{|T_m^{\mathcal{M}}|} - 1$. The minus one indicates the case in which no tool sets should be removed, and thus it is not necessary to use the TRM. For instance, in Example 3 with $T_1^{\mathcal{M}} = \{4, 2, 1\}$, the number of combinations is $2^{|3|} - 1 = 7$, namely {4}, {2}, {1}, {4, 2}, {4, 1}, {2, 1}, and {4, 2, 1}. The case {∅} in which there is no tool set to be removed is not taken into account (as explained). The cases {4}, {2}, and {1} can also be neglected, because removing only one tool set 4, 2, or 1 does not result in sufficient capacity to insert the required tool set 7. We propose an ILP model to determine the optimal solution for the tool replacement subproblem. To present the model, we define the binary variable λ_t equal to one if and only if tool set t in $T_m^{\mathcal{M}}$ is removed. The model is formulated as follows.

$$\min \sum_{t \in T_m^{\mathcal{M}}} sc_t \cdot \lambda_t \tag{23}$$

$$\text{s.t.:} \quad \sum_{t \in T_m^{\mathcal{M}}} \phi_t \cdot \lambda_t \geq \phi_m^{\mathcal{S}} \tag{24}$$

$$\lambda_t \in \{0, 1\} \qquad \forall t \in T_m^{\mathcal{M}} \tag{25}$$

The objective function (23) minimizes the total score of removed tool sets. Constraint (24) guarantees that the total size of removed tool sets has to be at least equal to the sufficient capacity $\phi_m^{\mathcal{S}}$. The main purpose here is that the model attempts to remove tool sets with the minimum total score because these tool sets might not be needed anymore or needed at latest among the present tool sets in the magazine. Consequently, it will keep tool sets required soonest, indicated by high given scores, hoping to reduce the number of tool switches and thus the tool setup time. Although the tool replacement subproblem is NP-hard (see Appendix A), the size of this subproblem is limited in practice at KMWE, and thus its optimal solution can be computed with little effort.

Example 5. From Examples 3 and 4, we have $T_1^{\mathcal{M}} = \{4, 2, 1\}$ and $sc_4 = 2$, $sc_2 = 1$ and $sc_1 = 0$. In addition, the sufficient capacity is $\phi_1^{\mathcal{S}} = \phi_{T_{61}} - (C - (\phi_4 + \phi_2 + \phi_1)) = 55 - (80 - (19 + 12 + 21)) = 27$. Then, by solving the ILP model, the result is $\lambda_4 = 0$, $\lambda_2 = 1$ and $\lambda_1 = 1$, which indicates that tool sets 1 and 2 are removed to obtain the sufficient capacity for the insertion of tool set 7 (required by operation (6,1)). The tool sets in the magazine are now updated to $T_1^{\mathcal{M}} = \{4, 7\}$.

5.7. Fitness evaluation

Fitness evaluation is to measure the objective function value subject to the constraints of the problem (Gen et al., 2008). In our matheuristic, fitness values of chromosomes are used for their ranking in the tournament and elitism selections described in Sections 5.3 and 5.5, respectively. The fitness value is equal to the total weighted tardiness of all operations and weighted tool setup time. The pseudocode of the fitness evaluation is presented in Algorithm 3.

In order to compute an individual's fitness value f_l, chromosome l is subsequently decoded from gene v_1 to v_n. For each gene v_g, the evaluation procedure first interprets job $v_g^{\mathcal{I}}$ ($v_g^{\mathcal{I}} = i | i \in I$) to define its corresponding operation (i, j) (line 3). If tool set T_{ij} required by the operation is present in the allocated machine, no tool switch is needed (lines 6–7). Otherwise, a tool switch is carried out and the present tool sets in the tool magazine are updated by using the TRM presented in Section 5.6 (lines 8–12). Next, the procedure calculates the ending time e_{ij} and tardiness Δd_{ij} of the operation and updates the available time $a_m^{\mathcal{M}}$ of its allocated machine (lines 14–20). The available time $a_m^{\mathcal{M}}$ is defined as the ending time of the last (most recent) operation on machine m. Finally, the fitness value of chromosome l is accumulated by adding the tardiness and tool setup time of operation (i, j).

Algorithm 3 Fitness evaluation.

Require: Chromosome l

$f_l \leftarrow 0$
for $g = 1$ to n **do**
  $(i, j) \leftarrow \text{interpret}(v_g^{\mathcal{I}})$
  $m \leftarrow v_g^{\mathcal{M}}$
  $t \leftarrow T_{ij}$
  if $\exists t \in T_m^{\mathcal{M}}$ **then**
    $z_{ijt} \leftarrow 0$ ▷ Case A
  else
    $z_{ijt} \leftarrow 1$ ▷ Case B
    $\phi_m^S \leftarrow \phi_{T_{ij}} - (C - \sum_{t \in T_m^{\mathcal{M}}} \phi_t)$
    $T_m^{\mathcal{M}} \leftarrow T_m^{\mathcal{M}} \setminus \{\text{TRM}(t \in T_m^{\mathcal{M}}) | \sum_{t \in T_m^{\mathcal{M}}} \phi_t \geq \phi_m^S\}$ ▷ Section 5.6
    $T_m^{\mathcal{M}} \leftarrow T_m^{\mathcal{M}} \cup T_{ij}$
  end if
  if $j = 1$ **then**
    $e_{ij} \leftarrow \max(r_{ij}, a_m^{\mathcal{M}}) + p_{ij} + \tau \cdot z_{ijt}$
  else
    $e_{ij} \leftarrow \max(r_{ij}, e_{i,j-1}, a_m^{\mathcal{M}}) + p_{ij} + \tau \cdot z_{ijt}$
  end if
  $a_m^{\mathcal{M}} \leftarrow e_{ij}$
  $\Delta d_{ij} \leftarrow \max(0, e_{ij} - d_{ij})$
  $f_l \leftarrow f_l + (w_d \cdot \Delta d_{ij} + w_s \cdot \tau \cdot z_{ijt})$
end for

Example 6. The two cases considered in the fitness evaluation are illustrated corresponding to the solution from Fig. 2 and using the information in Table 1. In addition, the tool setup time is supposed to be one hour ($\tau = 1$).

- Case A (no tool switch): $g = 2$, operation (2,1) is performed on machine 1 with tool set 4 and has a processing time $p_{ij} = 8$, release time $r_{ij} = 4$, and deadline $d_{ij} = 13$. Initially, machine 1 holds tool sets $T_1^{\mathcal{M}} = \{4, 2, 1\}$ at the beginning of the schedule, thus no tool setup time is incurred, i.e., $z_{ijt} = 0$. Consequently, the operation ends at $e_{ij} = \max\{4, 0\} + 8 + 1 \cdot 0 = 12$, which results in a tardiness $\Delta d_{ij} = 0$.
- Case B (tool switch): $g = 5$, operation (6,1) is performed on machine 1 with $T_{ij} = 7$, $p_{ij} = 6$, $r_{ij} = 25$, and $d_{ij} = 35$. As discussed in Example 5, this operation will cause a tool switch where the TRM is used to determine that tool sets 1 and 2 are removed to insert tool set 7, thus $z_{ijt} = 1$ and $T_1^{\mathcal{M}} = \{4, 7\}$. Consequently, the operation ends at $e_{ij} = \max\{25, 20\} + 6 + 1 \cdot 1 = 32$, which also results in a tardiness $\Delta d_{ij} = 0$.

6. Practitioner heuristic

In practice, scheduling of operations and tool switching on identical parallel machines are performed by a practitioner at the shop floor. The scheduling mainly relies on the practitioner's knowledge. It is because the scheduling department lacks the tool set tractability to take into account tool switches and tool setup time, whereas the practitioner is aware of tool sets present in the machines and thus may make a better arrangement. In this section, we capture the practitioner's knowledge in an approach, called practitioner heuristic (PH), to properly describe the current way of scheduling.

Let $T_m^{\mathcal{M}}$, m_T, M_C, ϕ_m^S, and $a_m^{\mathcal{M}}$ be defined as in Section 5. In addition, let m_P denote the earliest available machine and $\xi_m^{\mathcal{M}}$ denote the earliest starting time of an operation (i, j) on machine m, where $\xi_m^{\mathcal{M}} = \max(r_{ij}, a_m^{\mathcal{M}})$ if $j = 1$, otherwise $\xi_m^{\mathcal{M}} = \max(r_{ij}, e_{i,j-1}, a_m^{\mathcal{M}})$. The pseudocode of PH is presented in Algorithm 4.

The PH consists of two phases, i.e., initial tool allocations to machines (lines 2–11) and assignment and sequencing of operations

Algorithm 4 Practitioner heuristic.

Require: set of operations O, set of machines M, and set of tool sets T

1: $\hat{O} \leftarrow \text{sortEDD}(O)$
2: **for each** $(i, j) \in \hat{O}$ **do**
3: $m_T \leftarrow \{m \in M | T_{ij} \in T_m^{\mathcal{M}}\}$
4: **if** $\nexists m_T$ **then**
5: $M_C \leftarrow \{m \in M | C - \sum_{t \in T_m^{\mathcal{M}}} \phi_t \geq \phi_{T_{ij}}\}$
6: **if** $M_C \neq \emptyset$ **then**
7: $m^* \leftarrow \text{argmin}\{|T_m^{\mathcal{M}}| \ | m \in M_c\}$
8: $T_{m^*}^{\mathcal{M}} \leftarrow T_{m^*}^{\mathcal{M}} \cup T_{ij}$
9: **end if**
10: **end if**
11: **end for**
12: **for each** $(i, j) \in \hat{O}$ **do**
13: $m_P \leftarrow \text{argmin}(a_m^{\mathcal{M}} | m \in M)$
14: $m_T \leftarrow \{m \in M | T_{ij} \in T_m^{\mathcal{M}}\}$
15: **if** $\exists m_T$ **then**
16: **if** $m_P \neq m_T \wedge \xi_{m_T}^{\mathcal{M}} - \xi_{m_P}^{\mathcal{M}} \geq \Theta_m$ **then**
17: $m^* \leftarrow m_P$
18: $z_{ijt} \leftarrow 1$
19: $\phi_{m^*}^S \leftarrow \phi_{T_{ij}} - (C - \sum_{t \in T_{m^*}^{\mathcal{M}}} \phi_t)$
20: $T_{m^*}^{\mathcal{M}} \leftarrow T_{m^*}^{\mathcal{M}} \setminus \{\text{rand}(t \in T_{m^*}^{\mathcal{M}}) | \sum_{t \in T_{m^*}^{\mathcal{M}}} \phi_t \geq \phi_{m^*}^S\}$
21: $T_{m^*}^{\mathcal{M}} \leftarrow T_{m^*}^{\mathcal{M}} \cup T_{ij}$
22: **else**
23: $m^* \leftarrow m_T$
24: $z_{ijt} \leftarrow 0$
25: **end if**
26: **else**
27: **do** lines 17–21
28: **end if**
29: **if** $j = 1$ **then**
30: $e_{ij} \leftarrow \max(r_{ij}, a_{m^*}^{\mathcal{M}}) + p_{ij} + \tau \cdot z_{ijt}$
31: **else**
32: $e_{ij} \leftarrow \max(r_{ij}, e_{i,j-1}, a_{m^*}^{\mathcal{M}}) + p_{ij} + \tau \cdot z_{ijt}$
33: **end if**
34: $a_{m^*}^{\mathcal{M}} \leftarrow e_{ij}$
35: $\Delta d_{ij} \leftarrow \max(0, d_{ij} - a_{m^*}^{\mathcal{M}})$
36: **end for**
37: Objective value $\leftarrow w_d \cdot \sum_{(i,j) \in \hat{O}} \Delta d_{ij} + w_s \cdot \tau \cdot \sum_{(i,j) \in \hat{O}} \sum_{t \in T} z_{ijt}$

on machines (lines 12–37). Before placing initial tool sets into machines, all operations $(i, j) \in O$ are sorted in non-decreasing order of their due dates, i.e., the EDD rule, to create the sorted set $\hat{O}$ (line 1). Then, the initial tool sets in the machines are determined by allocating the required tool sets of sorted operations in $\hat{O}$. Each tool set is allocated to only one machine initially, and let the chosen machine be denoted as m^*. In other words, if any machine already holds a required tool set T_{ij}, then that tool set will not be allocated to any other machine at this phase. Otherwise, the tool set T_{ij} is placed into a machine in M_C which has enough remaining capacity. If there is more than one machine in M_C, the one holding the least number of tool sets in its magazine is selected (lines 5–9). If the tie still occurs, the lowest indexed machine is chosen. The first phase continues until all tool sets are allocated or when no machine can store any more tools.

In the second phase, the sorted operations in $\hat{O}$ are consecutively assigned to machines. The PH aims to allocate each operation (i, j) to either the earliest available machine m_P or the machine m_T which holds the required tool set T_{ij} (machines m_P and m_T are determined in lines 13 and 14, respectively). If there exists machine m_T and $m_T \neq m_P$, a trade-off between tardiness and tool setup time is considered. Choosing m_T may cause additional tardiness,

whereas choosing m_P implies additional tool setup time. The trade-off is performed by comparing the difference between the earliest starting time of a considered operation on two machines m_T and m_P with a threshold value Θ_m (lines 15–16). If $\xi^{\mathcal{M}}_{m_T} - \xi^{\mathcal{M}}_{m_P} \geq \Theta_m$, it is better to choose machine m_P, otherwise machine m_T is selected. The former indicates the need of a tool switch, i.e., $z_{ijt} = 1$, while the latter does not, i.e., $z_{ijt} = 0$. In order to perform a tool switch, the PH first determines the sufficient capacity $\phi^{\mathcal{S}}_{m^*}$ on the chosen machine m^* for the insertion of T_{ij}, then randomly remove a number of present tool sets to obtain at least $\phi^{\mathcal{S}}_{m^*}$ and place T_{ij} into the magazine (lines 19–21). If there does not exist machine m_T, then machine m_P is chosen and a tool switch will be carried out in the same manner (lines 26–27). Next, the available time of the chosen machine m^* and the tardiness of operation (i, j) are calculated (lines 29–35). The second phase of the PH is repeated until all operations $(i, j) \in \hat{O}$ are assigned. Finally, the PH computes the objective value of the obtained solution including total tardiness and tool setup time, which can be considered a reference point in the experiments.

Example 7. The PH is illustrated by using the data from Example 1. First, all operations in Table 1 are sorted in increasing order of their due dates, which results in the sorted set:

$\hat{O} = \{(1,1), (2,1), (3,1), (5,1), (7,1), (4,1), (3,2), (1,2), (6,1), (8,1)\}$

The PH then determines initial tool sets in two machines based on this order. Here, tool set 5 required by operation (1,1) is first considered. The magazines of both machines are empty at the beginning, thus there does not exist m_T, $M_C = \{1, 2\}$, and $|T^{\mathcal{M}}_1| = |T^{\mathcal{M}}_2| = 0$. Consequently, machine 1 is chosen because it has the lowest index, so $T^{\mathcal{M}}_1 = \{5\}$. Next, tool set 4 required by operation (2,1) is taken into account. Both machines do not hold tool sets 4 but still have enough remaining capacity, i.e., $M_C = \{1, 2\}$. In addition, $|T^{\mathcal{M}}_1| = 1$ while $|T^{\mathcal{M}}_2| = 0$. Hence, tool set 4 is placed into machine 2 because it has the least number of tool sets, so $T^{\mathcal{M}}_2 = \{4\}$. In a similar manner, the other tool sets are handled, which results in $T^{\mathcal{M}}_1 = \{1, 5\}$ and $T^{\mathcal{M}}_2 = \{2, 3, 4\}$. Note that tool sets 6 and 7 could not be placed into any machine due to the lack of capacity.

Second, the PH assigns each operation in $\hat{O}$ to one of the two machines. For operation (1,1), $m_P = 1$ since both machines have the same available time, i.e., $a^{\mathcal{M}}_1 = a^{\mathcal{M}}_2 = 0$, and thus machine with the lowest index is chosen. In addition, $m_T = 1$ because of $T_{11}(\text{tool set } 5) \in T^{\mathcal{M}}_1$. Hence, operation (1,1) is assigned to machine 1 without a tool setup, i.e., $m^* = 1$ and $z_{11t} = 0$ where $t = T_{11}$. We next take into account operation (6,1) since no machine is holding tool set 7 required by this operation until now. In this case, the earliest available machine is selected, i.e., $m^* = 2$ with $a^{\mathcal{M}}_2 = 31$ (as opposed to $a^{\mathcal{M}}_1 = 33$). After all operations in $\hat{O}$ are handled, the resulting assignments and sequences on the two machines are as follows:

$M_1: \quad (1,1) \succ (3,1) \succ (1,2) \succ (8,1)$

$M_2: \quad (2,1) \succ (5,1) \succ (7,1) \succ (4,1) \succ (3,2) \succ (6,1)$

7. Base cases and parameter tuning

This section describes four industry case studies at the KMWE company, followed by the parameter tuning of the proposed matheuristic. These case studies will be subsequently used for the computational experiments presented in Section 8.

7.1. Base cases: industrial case studies

We introduce four base cases, two of which originate from a work center having two identical parallel machines, while the other two are from another work center having six identical parallel machines. Information on the base cases is summarized in Table 3. Each base case has the name format |M|Mn, where |M| is the number of machines to process n operations. For example, 2M38 means that this case involves 2 machines and 38 operations. Each job in the base cases has at most two operations, i.e., at most one reentrant operation. The percent number of reentrant operations over all jobs is defined by ρ_R, with $\rho_R = |R|/|I|$, where $|R|$ is the number of reentrant operations. Note that one base case has a higher percent number of reentrant operations than the other in each work center. The average and standard deviation of the processing time shown in Table 3 are calculated from all operations of each base case. The table also provides the number of unique tool sets $|T|$ needed for processing n operations. It can be calculated that the tool ratio denoted by ρ_T, where $\rho_T = |T|/n$, is 0.5 for all base cases. Additionally, every machine has a magazine capacity of 80 tool sets, and the tool setup time of one hour is considered in all cases. In practice, the scheduling horizon of KMWE is four weeks. Hence, in alignment with the company's production scheduling, each base case covers the operations released within a period of four weeks. All data on the base cases is anonymized to protect confidentiality[1].

Table 3
Information on base cases.

Base cases	2-machine		6-machine	
	2M38	2M46	6M140	6M163
Nunmber of operations (n)	38	46	140	163
Number of reentrant operations ($\|R\|$)	7	4	20	2
Number of jobs ($\|I\|$)	31	42	120	161
% Reentrant operations (ρ_R)	22.58	9.52	16.67	1.24
Average processing time (hours)	5.91	7.02	10.76	13.68
Standard deviation of processing time (hours)	5.21	5.10	19.67	21.33
Number of unique tool sets ($\|T\|$)	19	23	70	82

Table 4
DoE factors and levels.

Factor	Description	Low (−)	High(+)
$\mathcal{B}$	Number of iterations applying POX	1	10
N_p	Population size	100	400
γ_2	Elitism selection rate	0.10	0.90
P_u	Probability of uniform mutation	0.01	0.20
P_s	Probability of swap mutation	0.01	0.20
γ_1	Tournament selection rate	0.05	0.20

7.2. Parameter tuning

The tuning of parameters is crucial for the effectiveness of a GA-based matheuristic's performance. For example, a small population size may result in less diversification in each generation while increasing of the population size consumes more computation time. However, it requires much effort to determine the best setting of GA parameters, because of several reasons such as mutual interactions between many parameters and dependency on the problem size (Gen et al., 2008). Therefore, the tuning of parameters in this paper is carried out to only provide a guideline for varying parameter values and identify which parameters have higher impact on the effectiveness of the proposed matheuristic. Although the parameter interactions may not be exactly the same for all problem sizes and parameter levels, it is still helpful to have an understanding of the interdependence and effects of parameters. The tuning of parameters is conducted with a 2^k factorial de-

[1] Data on base cases is available at https://github.com/vinhise/pmstr-basecases.

Table 5
Factorial design analysis.

Factor	$\mathcal{B}$		N_p		γ_2		P_u		P_s		γ_1	
	Obj.	C.T.	Obj.	C.T.	Obj.	C.T.	Obj.	C.T.	Obj.	C.T.	Obj.	C.T.
Low (−)	66.30	1590.89	66.34	451.83	65.75	1467.23	65.14	1565.68	65.52	1640.27	66.81	1185.58
High (+)	66.64	1535.13	66.59	2120.51	67.19	1105.12	67.80	1006.67	67.42	932.08	66.13	1386.77
Effect	0.34		0.25		1.44		2.66		1.91		−0.69	

Obj.: objective value, C.T.: computational time (seconds).

Table 6
Factorial design analysis of parameters $\mathcal{B}$, N_p, γ_2, P_s, γ_1 with low-level P_u.

Factor	$\mathcal{B}$		N_p		γ_2		P_s		γ_1	
	Obj.	C.T.	Obj.	C.T.	Obj.	C.T.	Obj.	C.T.	Obj.	C.T.
Low (−)	64.81	1458.60	64.97	662.06	63.63	1819.38	63.38	2198.86	66.06	1402.21
High (+)	65.47	1672.75	65.31	2469.30	66.66	1311.98	66.91	932.49	64.22	1729.15
Effect	0.66		0.34		3.03		3.53		−1.84	

Obj.: objective value, C.T.: computational time (seconds).

sign of experiments, where k is the number of factors (Box, Hunter, & Hunter, 2005). This design of experiments (DoE) consists of six factors (parameters) at two levels, low (−) and high (+), as shown in Table 4.

The parameters in Table 4 are tuned on two cases originating from the 6-machine work center. The information about these cases are summarized in Appendix B[2]. In the tuning phase, the stopping criteria of the matheuristic, i.e., maximum computational time *maxTime* and no-improvement consecutive generations G_c are set to 3600 seconds and 20, respectively. There are a total of 64 (2^6) different sets of parameter values in the DoE. Table 5 shows the effect of each factor on the results of the experiments. The effect is calculated by $E_b^a = \bar{y}_+ - \bar{y}_-$ where $\bar{y}_+$ and $\bar{y}_-$ are the average of b (i.e., objective value) for the sets of parameter values in which factor a is at the high (+) and low (−) level, respectively (Box et al., 2005). For each factor, the average objective value and computational time at the low and high levels are also presented in Table 5.

One can note that due to the minimization nature of the problem, a lower objective value is preferred over a higher one, and therefore a positive effect indicates that setting a factor to the low level (−) is more beneficial and vice versa. In Table 5, the probability of the uniform mutation P_u produces the most considerable effect among all factors while the effects of the other five factors are quite moderate. For P_u, setting to the low level improves the result (decreases the objective value). Hence, we set P_u to the low level, i.e., 0.01, in the remaining experiments in this paper.

When the value of P_u is chosen, the interactions between the remaining factors and the chosen level of P_u are considered. The results are shown in Table 6. When P_u is at the low level, the effects of the swap mutation probability P_s and elitism selection rate γ_2 are distinctive in comparison with the other three remaining factors. And setting P_s and γ_2 to the low level (−) is more preferable because of the improvement on the objective value at this level. Hence, at this stage the setting of parameters can be $P_u = 0.01$, $P_s = 0.01$, and $\gamma_2 = 0.1$. The interactions between these three factors and the remaining ones are next considered in Table 7.

According to Table 7, when P_u, P_s, and γ_2 are at the low level, the effects of the number of iterations applying POX ($\mathcal{B}$) and tournament selection rate (γ_1) become distinctive and considerable. We can see that setting $\mathcal{B}$ to the low level (−) has a positive effect while setting γ_1 to the high level (+) is more beneficial. For the last parameter, the population size N_p, although there is no major difference between the objective values obtained at the low and high levels, the computational time at the low level is about two times faster than that at the high one. Therefore, it is more effective to set N_p to the low level. In summary, all parameters are set at the low level, except the tournament selection rate set at the high level, i.e., $\mathcal{B} = 1$, $N_p = 100$, $\gamma_2 = 0.1$, $P_u = 0.01$, $P_s = 0.01$, and $\gamma_1 = 0.20$.

Table 7
Factorial design analysis of parameters $\mathcal{B}$, N_p, γ_1 with low-level P_u, γ_2, P_s.

Factor	$\mathcal{B}$		N_p		γ_1	
	Obj.	C.T.	Obj.	C.T.	Obj.	C.T.
Low (−)	57.75	2531.48	60.13	1770.39	62.75	2340.84
High (+)	63.13	2966.72	60.75	3727.81	58.13	3157.36
Effect	5.38		0.63		−4.63	

Obj.: objective value, C.T.: computational time (seconds).

8. Computational experiments

In this section, we evaluate the performance of the MILP model, the practitioner heuristic (PH), and the matheuristic (MH). The evaluation consists of two parts. The first part compares the performance of the approaches to demonstrate the effectiveness of the proposed MH. The second part carries out the sensitivity analysis to generate managerial insights for practitioners. For most of this section, we present the results of case 6M140 to show representative trends of the outcome. Then, we summarize the results of a sensitivity analysis for all of the base cases in Section 8.2.4. Note that case 6M140 is selected since it represents a large-sized instance, which has a high number of machines, operations, reentrant operations, and unique tool sets.

In the experiments of this section, the MH stops after it reaches the maximum computational time *maxTime* of 3600 seconds or the best solution is not improved over G_c of 20 consecutive generations as set in the parameter tuning section. In practice, the scheduling department of KMWE has 6 hours, from 12 midnight to 6 in the morning, to compute the solutions. In this paper, we restrict the computational time to one hour due to limited computational resources as well as to provide more flexibility if the department wants to do rescheduling. For the PH, the threshold value Θ_m is set to 72 hours (3 days) as in practice. Additionally, the weight coefficients of the tardiness w_d and tool setup times w_s in the objective function (or fitness evaluation) are both set to 1, i.e., $w_d = w_s = 1$.

[2] Data on the cases for the parameter tuning is available at https://github.com/vinhise/pmstr-basecases.

Table 8
Comparison of performance of the proposed approaches for case 6M140.

n	MILP				MILP-SB				PH				MH				Gap in Obj. (%)	
	Obj.		C.T.		Obj.		C.T.		Obj.		C.T.		Obj.		C.T.		MH vs. MILP-SB	MH vs. PH
	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ		
15	*0.00	0.00	6.96	0.99	*0.00	0.00	5.02	0.26	0.00	0.00	0.04	0.04	0.00	0.00	15.57	3.81	0.00	0.00
25	9.00	0.00	3600.30	0.02	4.10	0.32	3600.26	0.03	0.00	0.00	0.01	0.00	0.00	0.00	14.58	0.86	−100.00	0.00
30	22.50	1.58	3600.26	0.02	17.00	0.00	3600.23	0.01	0.00	0.00	0.01	0.00	0.00	0.00	16.07	1.09	−100.00	0.00
60	–	–	–	–	–	–	–	–	15.30	0.67	0.03	0.02	10.00	0.67	113.94	43.62	–	−34.64
90	–	–	–	–	–	–	–	–	32.00	1.94	0.02	0.01	22.20	1.48	429.85	288.04	–	−30.63
120	–	–	–	–	–	–	–	–	55.90	2.23	0.17	0.13	37.47	1.04	2286.18	802.98	–	−32.97
140	–	–	–	–	–	–	–	–	62.30	2.16	0.06	0.02	45.60	2.55	1981.91	731.41	–	−26.81

Obj.: objective value, C.T.: computational time (s), μ: mean, σ: standard deviation.
(*) indicates the optimal solution found by the MILP models.

Moreover, the MH and PH are programmed in the MATLAB®, while the MILP model is written in the optimization programming language (OPL) and solved by the IBM® ILOG® CPLEX® Optimization Studio V12.8. All the experiments have run on a PC having an Intel® Core i7-4700MQ, 2.40 gigahertz, and 8 gigabyte of RAM.

8.1. Comparison of the proposed approaches

The comparison is performed on base case 6M140, from which six problem instances are generated. These instances are constructed as follows. All operations in the base case are sorted in non-decreasing order of their release times, and then the first n operations in the sorted order are taken to generate a problem instance. With this manner, we construct the six problem instances by subsequently increasing the number of first operations in the range of 15–140.

Two MILP models are considered in this experiment. The first model, named MILP, does not have the symmetry-breaking constraint (22), whereas the second model, named MILP-SB, contains this constraint. The performance of the proposed approaches is presented in Table 8. To ensure fair comparisons, the maximum computational time of the MILP models and PH are also set to 3600 seconds as in the MH. Additionally, each approach runs 10 times to calculate the mean and standard deviation of the objective value and computational time. A percentage gap (%Gap) in Table 8 is calculated from the mean objective values found by two respective approaches. The minus numbers indicate that the mean objective values obtained by the MH are smaller (better) than those of the PH or MILP models and vice versa.

Table 8 shows that for all instances, the MH performs equally good or better than the MILP, MILP-SB, and PH in terms of the objective value obtained within the maximum computational time. Both MILP models can obtain the optimal solution for the first instance of 15 operations, where the zero value in the objective means that there is no tardy job and no tool switch. Nevertheless, they can only find feasible solutions for the next two instances. Although the MILP-SB improves the objective value for two out of the three instances as compared with the MILP owing to the symmetry-breaking constraint (22), they both struggle to solve the problem when increasing the number of operations, even with the second instance of 25 operations. Both MILP models also cannot find any feasible solutions for the remaining four instances.

The MH and PH, by contrast, are able to find feasible solutions for all problem instances. The MH also finds the optimal solution for the first instance, but it produces better results than the MILP-SB (and, of course, the MILP) for the second and third instances. Moreover, we can see that both MH and PH obtain the same results in the first three instances, but there is a reduction in the objective value gained by MH over PH, which fluctuates in the range of 26–35% for the larger cases. Additionally, the standard deviation of the objective value found by the MH is quite small in comparison to the mean. In general, these results prove that the proposed MH performs well, and it is crucial that the MH can improve solution quality by about 30% on average as opposed to the current way of scheduling (PH) used at the company within a reasonable time of one hour.

8.2. Sensitivity analysis

This section considers the base cases generated from our industry partner and focuses on producing managerial insights for practitioners. Specifically, we compare the performance of the MH and PH. This allows us to quantify the potential room for improvement in current practice. The MILP and MILP-SB are excluded from the analysis, since they were not able to obtain a feasible solution for the 6-machine cases or their solutions are significantly worse than those of the MH and PH for the 2-machine cases (see Appendix C for more details). We here consider three scenarios for the sensitivity analysis as follows:

- Scenario 1 (*tool ratio*): an increase in the tool ratio (i.e., the number of tool sets required to produce operations) implies a move towards a high-mix low-volume production environment where more operations require unique tool sets. Thus, we vary the tool ratio in the base case to examine the performance of the MH compared to the PH.
- Scenario 2 (*tool magazine capacity*): the size of the tool magazine is a critical parameter affecting the tool setup time. Therefore, the tool magazine sizes of machines in the base case are varied to investigate how both approaches would react to different capacities. Results of this analysis could be useful for capacity planning in practice.
- Scenario 3 (*shifting deadlines*): shifting of job deadlines is not uncommon in the industry, especially with “hot” jobs. Hence, due dates of operations in the base case are shifted to study the performance of the MH relative to the PH in case of reduced or extended deadlines.

We present the results of case 6M140 to show the outcome trends of these scenarios in Sections 8.2.1, 8.2.2, and 8.2.3. The results of the other base cases are presented in Appendix D. Finally, we discuss the summarized results of all the base cases in Section 8.2.4.

8.2.1. Tool ratio

We change the number of tool sets that is needed to process operations in the base cases. This results in a change in the tool ratio ρ_T. From each base case, with $\rho_T = 0.5$, we create four more instances, two of which have smaller tool ratios while the other two have bigger tool ratios than that base case. Table 9 presents five different instances, generated from case 6M140, in which ρ_T

Table 9
Performance of MH in comparison to PH when varying tool set ratio.

ρ	$\|T\|$	PH				MH				Gap in Obj.	
		Obj.		C.T.		Obj.		C.T.		Time unit (hours)	%Gap
		μ	σ	μ	σ	μ	σ	μ	σ		
0.26	36	19.20	2.39	0.19	0.20	13.50	1.84	486.42	209.57	−5.70	−29.69
0.38	53	43.50	3.06	0.05	0.01	29.90	2.81	1505.33	708.86	−13.60	−31.26
0.50	70	62.30	2.16	0.06	0.02	45.60	2.55	1981.91	731.41	−16.70	−26.81
0.63	88	81.30	1.70	0.09	0.02	59.30	1.57	2356.93	663.93	−22.00	−27.06
0.75	105	95.10	2.13	0.10	0.03	71.90	1.60	1880.04	724.61	−23.20	−24.40

Obj.: objective value, C.T.: computational time (s), μ: mean, σ: standard deviation.

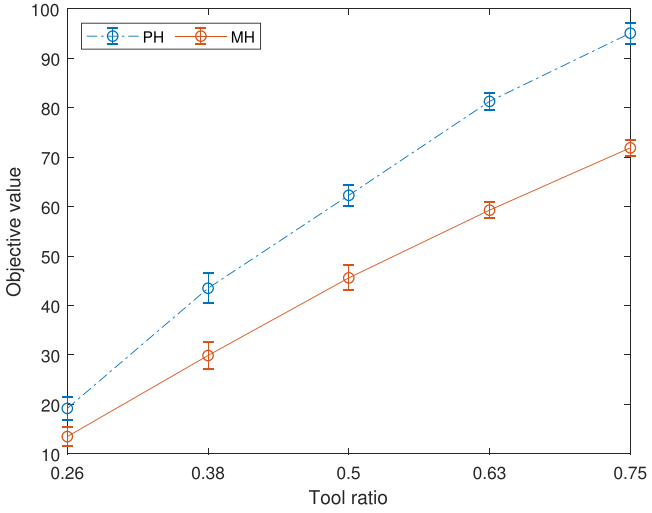

Fig. 7. Objective value when varying tool ratio.

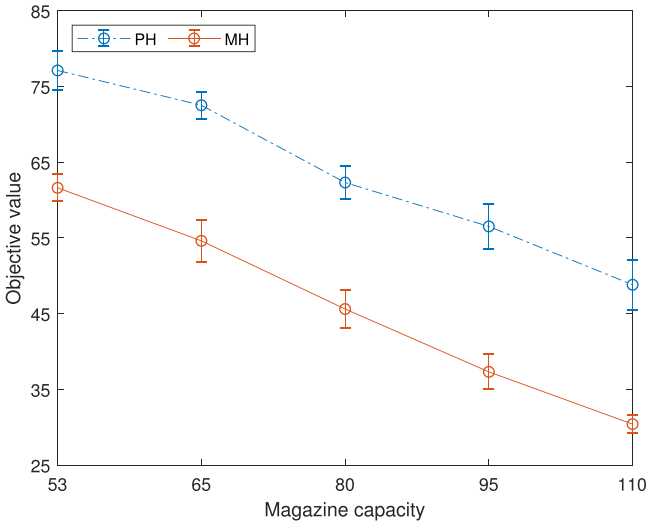

Fig. 8. Objective value when varying tool magazine capacity.

is changed from 0.26 to 0.75 by varying $|T|$ from 36 to 105 (see columns 1–2)[3].

The performance of the MH in comparison to the PH when varying the tool ratio for the five instances of case 6M140 is shown in Table 9 and Fig. 7 (see Appendix D.1 for this scenario's results of the other base cases). For each problem instance, we perform 10 runs for the MH and PH to calculate the mean and standard deviation of the objective value and computational time. In addition to the percentage gap (%Gap), Table 9 presents the gap in terms of time unit (hours), which is calculated by subtracting the mean objective value of the PH from that of the MH. Note that the gap can be expressed in time unit, i.e., hour, because an objective value indicates a total hour of tardiness and tool setup, which results from the weight coefficients $w_d = w_s = 1$. Fig. 7 plots the mean objective values in Table 9, together with the standard deviations marked as the error bars.

It can be seen that the mean objective values of both MH and PH increase with an increase in the tool ratio. It is because the requirement of more tool sets implies more tool setup time and thus more tardiness. Additionally, the difference (gap expressed in hours) between the MH and PH becomes bigger, i.e., from about 6 to 24 hours, when the tool ratio increases from 0.26 to 0.75. This indicates that the company, with our MH, can obtain a reduction of 6 to 24 hours in the total tardiness and tool setup time. The average percentage gap (improvement) obtained by our MH is 27.84%. These results demonstrate the good performance of the proposed MH regardless of changes in the tool ratio, and its application becomes more attractive when moving towards a high-mix low-volume production environment.

8.2.2. Tool magazine capacity

This scenario evaluates the performance of the MH relative to the PH when changing tool magazine capacity. Five instances including a base case are considered. Each base case with the magazine capacity of 80 is chosen as the middle instance. Among the other four, two have less and the remaining two have more capacity than the base case. The minimum capacity is equal to the largest tool set size, i.e., 53, since the tool magazine of any machine must at least be able to hold a tool set with such size. The five different capacities C, which range from 53 to 110, are shown in column 1 of Table 10. The other column headings are the same as explained in Section 8.2.1. Each approach also runs 10 times for each instance. The results for the five instances of case 6M140 are presented in Table 10 and Fig. 8 (see Appendix D.2 for the results of the other base cases in this scenario)[4].

The mean objective values of the MH and PH decrease with an increase in the magazine capacity. This is obtained from less tool setup time, because more tool sets can be stored and more tool sets are allowed to be kept in the magazine as a tool switch occurs. In addition, the percentage gap between the two approaches has an increasing trend, and on average, the improvement gained by the MH is around 29%. On the other hand, the gap expressed in hours seems to fluctuate and has no visible trend. This fluctuation may imply that the MH can gain more from the increasing magazine capacity than the PH for some instances and vice versa for others. For further analysis, we hence calculate the average reduction of the objective value per increase of the capacity, denoted by η, for each approach according to Eq. (26). In this equation, μ_k

[3] Data for all instances of all base cases in Scenario 1 is available from https://github.com/vinhise/pmstr-basecases.

[4] Data for all instances of all base cases in Scenario 2 is available from https://github.com/vinhise/pmstr-basecases.

Table 10
Performance of MH in comparison to PH when varying magazine capacity.

C	PH				MH				Gap in Obj.	
	Obj.		C.T.		Obj.		C.T.		Time unit (hours)	%Gap
	μ	σ	μ	σ	μ	σ	μ	σ		
53	77.10	2.56	0.06	0.02	61.60	1.78	2612.96	899.28	−15.50	−20.10
65	72.50	1.78	0.06	0.01	54.60	2.76	2038.17	742.53	−17.90	−24.69
80	62.30	2.16	0.06	0.02	45.60	2.55	1981.91	731.41	−16.70	−26.81
95	56.50	3.03	0.06	0.02	37.30	2.31	2216.79	897.14	−19.20	−33.98
110	48.80	3.29	0.05	0.02	30.40	1.17	1621.35	346.06	−18.40	−37.70

Obj.: objective value, C.T.: computational time (s), μ: mean, σ: standard deviation.

Table 11
Performance of MH in comparison to PH when shifting deadlines.

Shift (days)	PH				MH				Gap in Obj.	
	Obj.		C.T.		Obj.		C.T.		Time unit (hours)	%Gap
	μ	σ	μ	σ	μ	σ	μ	σ		
−14	115.62	5.45	0.07	0.03	58.30	2.58	2581.52	987.70	−57.32	−49.58
−7	72.00	2.16	0.06	0.02	47.00	1.76	2918.43	503.28	−25.00	−34.72
0	62.30	2.16	0.06	0.02	45.60	2.55	1981.91	731.41	−16.70	−26.81
+7	66.50	2.76	0.06	0.02	47.00	3.02	2434.19	1080.87	−19.50	−29.32
+14	67.20	2.49	0.07	0.03	46.90	2.08	1943.72	718.27	−20.30	−30.21

Obj.: objective value, C.T.: computational time (s), μ: mean, σ: standard deviation.

and C_k denote the mean objective value and magazine capacity of instance k, respectively.

$$\eta = \frac{\sum_{k=1}^{4} \left(\frac{\mu_{k+1} - \mu_k}{C_{k+1} - C_k} \right)}{4} \tag{26}$$

According to Eq. (26), $\eta = -0.55$ for MH, while $\eta = -0.49$ for PH. This indicates that the MH is better capable of dealing with changes in the magazine capacity. In general, the results in this scenario provide more persuasive evidence of the improvement obtained from our MH and its better utilization of the magazine capacity than the PH used in the company.

8.2.3. Shifting deadlines

This scenario focuses on reducing and extending the due dates of operations d_{ij}. We construct five instances in which the third one is a base case with a shift of 0 days. The remaining instances have shifts within a margin of the average duration between the release time r_{ij} and the due date d_{ij} of all operations in each base case. This average duration is around 14 days for the cases having 6 machines and 2 days for the cases having 2 machines. Hence, the due dates of operations in the remaining instances may be reduced (−) or extended (+) up to 14 days for case 6M140 as shown in the first column of Table 11. Also, it seems logical to shift all operations of a job i, if $n_i > 1$, instead of shifting one individual operation. Here around 19–20% number of jobs in each base case (e.g., 25 jobs in case 6M140) is randomly chosen for which their operations' due dates are shifted. It is notable that we change the due dates of the same operations in all four remaining instances for consistency. The results of case 6M140 are shown in Table 11 and Fig. 9 (see Appendix D.3 for the results of the other base cases in this scenario)[5].

When the due dates are reduced, it is likely to result in tight deadlines of those operations. This causes an upward trend in the objective values of both approaches. For the extreme case, instance 1, where the due dates are reduced by 14 days, our MH obtains the biggest percentage of improvement (gap), i.e., about 50% or 60 hours (2.5 days). The high objective values in this instance are due to very tight deadlines, each chosen operation should be processed as soon as possible to prevent tardiness. Any delay, e.g., a tool setup or no available machine, may cause an increase in the objective value. This effect decreases substantially when the reduced deadlines are not very tight, i.e., there is more slack, as shown in the second instance (−7 days).

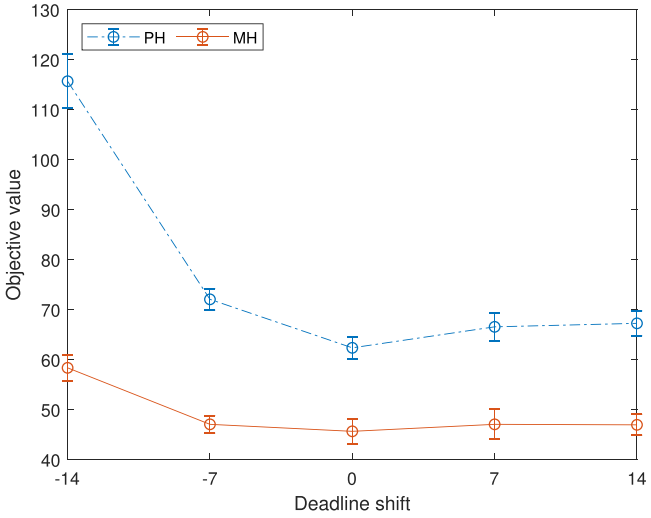

Fig. 9. Objective value when shifting deadlines.

On the other hand, when the operations' due dates are extended, the improvement obtained by MH over PH slightly fluctuates between 26% and 30%. One can note that the mean objective value of the MH slightly increases when extending deadlines. It is because some of the operations requiring the same tool sets, due to deadline shifts, may be sequenced quite far away from each other, which may result in additional tool setup. These results confirm the benefit of using MH when there are changes in the due dates of operations.

8.2.4. Discussion on all base cases

This section discusses the results of sensitivity analysis for the other bases cases, i.e., 2M38, 2M46, and 6M163, and summarizes them together with those of case 6M140. Detailed results of the other base cases can be seen in Appendix D (Figs. 10–12 and

[5] Data for all instances of all base cases in Scenario 3 is available from https://github.com/vinhise/pmstr-basecases.

Table 12
Summary of results of sensitivity analysis for all base cases.

Scenario	2-machine cases	Gap in Obj.		6-machine cases	Gap in Obj.	
		Time unit (hours)	%Gap		Time unit (hours)	%Gap
Tool ratio (scenario 1)	2M38	−7.18	−44.87	6M140	−16.24	−27.84
	2M46	−10.48	−41.22	6M163	−22.44	−26.72
	Average	*−8.83*	*−43.04*		*−19.34*	*−27.28*
Magazine capacity (scenario 2)	2M38	−6.74	−41.84	6M140	−17.54	−28.66
	2M46	−10.20	−37.87	6M163	−17.90	−23.01
	Average	*−8.47*	*−39.85*		*−17.72*	*−25.84*
Deadline shift (scenario 3)	2M38	−8.08	−43.43	6M140	−27.76	−34.13
	2M46	−12.80	−41.13	6M163	−59.34	−35.13
	Average	*−10.44*	*−42.28*		*−43.55*	*−34.63*
All scenarios	*Average*	*−9.25*	*−41.73*		*−26.87*	*−29.25*

Tables 15–17). The summary of all cases' results is shown in Table 12. The values in this table are the average difference in hours and average percentage gap of all instances in each base case obtained by MH over PH for each scenario. The italic values indicate the average of all 2- or 6-machine cases for each scenario and for all scenarios.

We observe a similar trend of results in each scenario of the other base cases when compared to the corresponding scenario of case 6M140, except in scenario 1 (tool ratio) for cases 2M38 and 2M46. The percentage gap tends slightly downwards in the 2-machine cases of this scenario as opposed to more or less constant in the 6-machine ones (see Table 15 in Appendix D). However, the margin of improvement is still considerable with 34% minimum and 43% on average. Another observation is that when dealing with the cases having a higher percentage of the number of reentrant operations ρ_R (e.g., 2M38 ($\rho_R = 22.6\%$) versus 2M46 ($\rho_R = 9.5\%$) or 6M140 ($\rho_R = 16.7\%$) versus 6M163 ($\rho_R = 1.2\%$)), the average percentage of improvement almost remains the same (e.g., for scenario 1, 44.87% (2M38) versus 41.22% (2M46) or 27.84% (6M140) versus 26.72% (6M163)).

In general, the average improvements for all scenarios in the 2- and 6-machine cases are approximately 40% and 30%, respectively. Additionally, it is worth spending one hour of computational time for our MH to obtain a reduction of around 10 to 24 hours of the total tardiness and tool setup time on average, although the PH takes much less time to generate a solution. In particular, when there are "hot" jobs (scenario 3), the MH can save about 0.5–2.5 days, and even up to 8 days/196 hours (see Table 17 in Appendix D, case 6M163, instance (−14)).

To summarize the sensitivity analyses, the proposed MH is able to produce a considerable amount of improvement as compared to the current way of scheduling within a computational time acceptable in practice. It is also important that the improvement obtained by our MH tends to increase when the production continues to move towards a high-mix low-volume environment or when there is more tool magazine capacity or more operations with tight deadlines. The proposed MH is also robust when production has more precedence relations between operations.

9. Conclusions

This paper studies the problem of parallel machine scheduling with tool replacements in an FMS work center. The work is inspired from the high-mix low-volume production environment of the KMWE company. The main novelty of the problem lies in the consideration of new characteristics including the reentrance of jobs, release times of operations, non-uniform tool set sizes, and multiple objectives of minimizing the tardiness and tool setup time. An MILP model was formulated for the problem. However, the model could only be applied to small-scale problems (e.g., up to 25 operations). Therefore, we developed a new matheuristic approach combining a GA and ILP model to find good-quality solutions within a reasonable computation time in practice.

The computational experiments on the industry case studies reveal that the proposed MH outperforms not only the MILP but also the PH, i.e., current way of scheduling, used at the company. To fully evaluate the effectiveness of the proposed MH, we performed sensitivity analyses on three parameters, such as tool ratio, tool magazine capacity, and due date. On average, our analyses indicate a 20–40% improvement in current practice by using the proposed MH approach.

Interesting directions for future research include the extension of our solution approach to a rolling horizon schedule, the inclusion of preemptions in the processing of operations and constraints on the availability of operators who are responsible for tool setup activities, and the extension to multiple work centers in a flexible job shop environment.

Acknowledgments

This research work has been funded by a grant provided by Provincie Noord-Brabant for the project "Advanced Manufacturing Logistics". We also thank Koen Herps for his support in the case studies.

Appendix

Appendix A. Proof of the tool replacement subproblem

Considering the ILP model (23)–(25) proposed in Section 5.6, let Φ denote the total size of all tool sets in $T_m^{\mathcal{M}}$, so $\Phi = \sum_{t \in T_m^{\mathcal{M}}} \phi_t$. Then the optimal solution of the subproblem is the complement of the set of tool sets found for:

$$\max \sum_{t' \in T_m^{\mathcal{M}}} sc_{t'} \cdot \lambda_{t'} \tag{27}$$

$$\text{s.t.:} \quad \sum_{t' \in T_m^{\mathcal{M}}} \phi_{t'} \cdot \lambda_{t'} \leq \Phi - \phi_m^S \tag{28}$$

$$\lambda_{t'} \in \{0, 1\} \qquad \forall t' \in T_m^{\mathcal{M}} \tag{29}$$

This is exactly the knapsack problem. In other words, the subproblem is NP-hard.

Appendix B. Information of cases for parameter tuning

The information about the cases that are used for parameter tuning is presented in Table 13.

Table 13
Information on cases used for parameter tuning.

Parameter tuning cases	6M124	6M183
Number of operations ($\|O\|$)	124	183
Number of reentrant operations ($\|RO\|$)	20	16
Number of jobs ($\|J\|$)	104	167
% Reentrant operations (ρ_R)	19.23	9.58
Average processing time (hours)	11.91	12.26
Standard deviation of processing time (hours)	13.13	19.69
Number of unique tools ($\|T\|$)	71	83

Appendix C. Performance of the approaches for all base cases

The results of the base cases with respect to different approaches are shown in Table 14.

Table 14
Performance of the approaches for all base cases.

BC	n	MILP				MILP-SB				PH				MH			
		Obj.		C.T.		Obj.		C.T.		Obj.		C.T.		Obj.		C.T.	
		μ	σ	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ	μ	σ
2M38	38	7503.38	486.91	3600.18	0.02	5374.89	0.00	3600.35	0.19	16.70	1.42	0.01	0.00	10.10	0.32	101.68	25.61
2M46	46	13063.29	0.00	3600.28	0.01	11650.14	0.00	3600.26	0.02	28.20	1.99	0.02	0.01	17.10	0.32	272.03	54.38
6M140	140	–	–	–	–	–	–	–	–	62.30	2.16	0.06	0.02	45.60	2.55	1981.91	731.41
6M163	163	–	–	–	–	–	–	–	–	79.90	2.51	0.07	0.02	59.50	2.17	2941.53	866.83

Note that the MILP does not contain the symmetry-breaking constraint (22), whereas the MILP-SB includes this constraint. Moreover, in all tables in the appendices, Obj. and C.T. represent the objective value and the computational time while μ and σ indicate the mean and standard deviation, respectively.

Appendix D. Results of three scenarios of other cases

We present the results of the other base cases: 2M38 and 2M46 (2-machine work center) and 6M163 (6-machine work center) in three scenarios, including the variation of tool ratio, magazine capacity, and deadline shift in Sections D.1, D.2, and D.3, respectively. We also provide the data for the instances in all scenarios[6]. Each data (file) has the name format as follows:

[*number of machines*]M[*number of operations*]_[*tool ratio/capacity/number of shifted days*]R/C/D where R, C, and D represent the instances in which the tool ratio, magazine capacity, and due dates are varied, respectively. For example, 2M38_0.5R indicates that the instance has 2 machines, 38 operations, and the tool ratio equal to 0.5, or 6M163_-14D is the instance having 6 machines, 163 operations, and due dates (of around 20% of the number of jobs) reduced by 14 days.

D.1. Variation of tool ratio

[6] Data for all instances of all base cases in all scenarios is available from https://github.com/vinhise/pmstr-basecases.

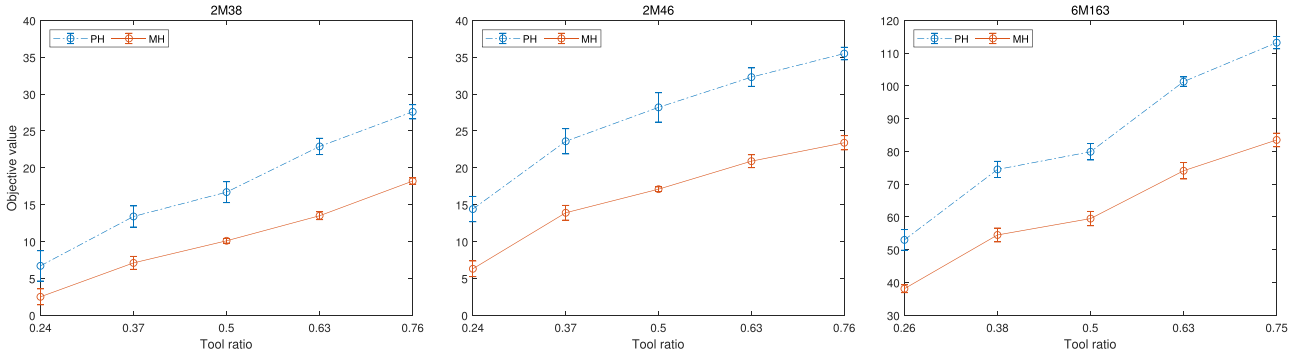

Fig. 10. Objective value when varying tool ratio in cases 2M38, 2M46, and 6M163.

Table 15
Performance of MH in comparison to PH when varying tool set ratio ρ in cases 2M38, 2M46, and 6M163.

Case	ρ	$\|T\|$	PH				MH				Gap in Obj.	
			Obj.		C.T.		Obj.		C.T.		Time unit (hours)	%Gap
			μ	σ	μ	σ	μ	σ	μ	σ		
2M38	0.24	9	6.70	2.06	0.07	0.08	2.50	1.08	94.81	38.19	−4.20	−62.69
	0.37	14	13.40	1.43	0.01	0.01	7.10	0.88	113.42	30.44	−6.30	−47.01
	0.50	19	16.70	1.42	0.01	0.00	10.10	0.32	101.68	25.61	−6.60	−39.52
	0.63	24	22.90	1.10	0.02	0.01	13.50	0.53	83.65	22.44	−9.40	−41.05
	0.76	29	27.60	0.97	0.01	0.00	18.20	0.42	61.14	15.52	−9.40	−34.06
2M46	0.24	11	14.40	1.71	0.05	0.05	6.30	1.06	230.03	65.18	−8.10	−56.25
	0.37	17	23.60	1.71	0.02	0.01	13.90	0.99	287.20	83.90	−9.70	−41.10
	0.50	23	28.20	1.99	0.02	0.01	17.10	0.32	272.03	54.38	−11.10	−39.36
	0.63	29	32.30	1.25	0.02	0.01	20.90	0.88	278.73	84.98	−11.40	−35.29
	0.76	35	35.50	0.85	0.02	0.01	23.40	0.97	221.00	96.81	−12.10	−34.08
6M163	0.26	42	53.00	3.16	0.15	0.14	38.10	1.20	2223.20	821.31	−14.90	−28.11
	0.38	62	74.50	2.46	0.06	0.02	54.50	2.12	3015.74	878.78	−20.00	−26.85
	0.50	82	79.90	2.51	0.07	0.02	59.50	2.17	2941.53	866.83	−20.40	−25.53
	0.63	102	101.30	1.42	0.08	0.02	74.10	2.51	2922.07	702.77	−27.20	−26.85
	0.75	122	113.20	1.87	0.07	0.02	83.50	2.07	2477.92	935.02	−29.70	−26.24

D.2. Variation of magazine capacity

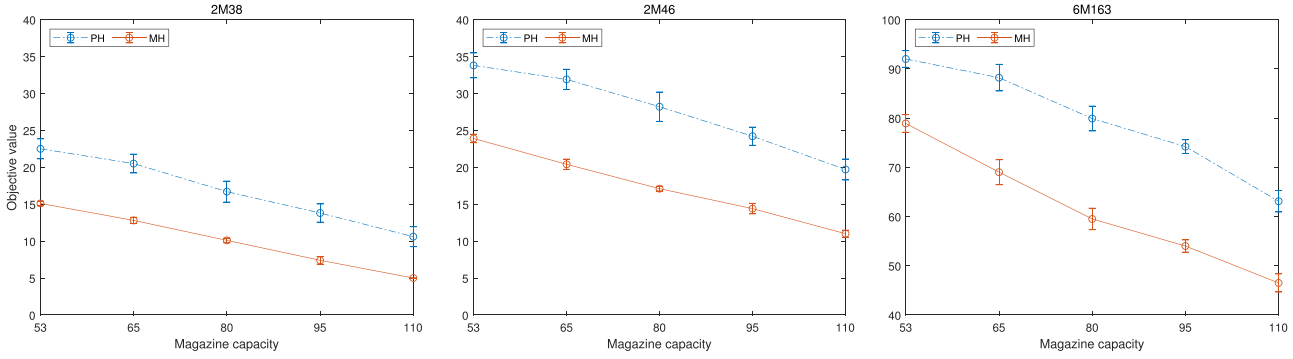

Fig. 11. Objective value when varying magazine capacity in cases 2M38, 2M46, and 6M163.

Table 16
Performance of MH in comparison to PH when varying magazine capacity C in cases 2M38, 2M46, and 6M163.

Case	C	PH				MH				Gap in Obj.	
		Obj.		C.T.		Obj.		C.T.		Time unit (hours)	%Gap
		μ	σ	μ	σ	μ	σ	μ	σ		
2M38	53	22.50	1.35	0.08	0.09	15.10	0.32	266.34	73.69	−7.40	−32.89
	65	20.50	1.27	0.01	0.01	12.80	0.42	158.64	38.66	−7.70	−37.56
	80	16.70	1.42	0.01	0.00	10.10	0.32	101.68	25.61	−6.60	−39.52
	95	13.80	1.23	0.01	0.00	7.40	0.52	93.58	31.81	−6.40	−46.38
	110	10.70	1.49	0.01	0.00	5.00	0.00	47.38	9.72	−5.70	−53.27
2M46	53	33.80	1.69	0.08	0.08	23.90	0.57	380.40	132.64	−9.90	−29.29
	65	31.90	1.37	0.02	0.01	20.40	0.70	429.15	142.69	−11.50	−36.05
	80	28.20	1.99	0.02	0.01	17.10	0.32	272.03	54.38	−11.10	−39.36
	95	24.20	1.23	0.02	0.01	14.40	0.70	214.51	77.65	−9.80	−40.50
	110	19.70	1.42	0.01	0.00	11.00	0.47	169.71	55.23	−8.70	−44.16
6M163	53	92.00	1.70	0.17	0.15	78.90	1.79	1938.89	769.93	−13.10	−14.24
	65	88.20	2.70	0.07	0.02	69.00	2.54	2777.87	699.41	−19.20	−21.77
	80	79.90	2.51	0.07	0.02	59.50	2.17	2941.53	866.83	−20.40	−25.53
	95	74.20	1.40	0.06	0.02	54.00	1.25	2540.28	889.93	−20.20	−27.22
	110	63.10	2.18	0.05	0.01	46.50	1.84	2603.03	647.44	−16.60	−26.31

D.3. Variation of due dates

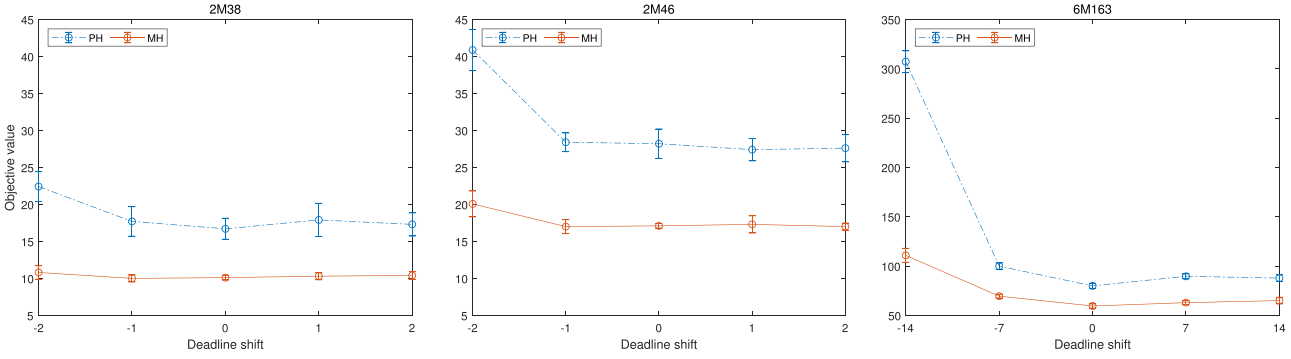

Fig. 12. Objective value when shifting deadlines in cases 2M38, 2M46, and 6M163.

Table 17
Performance of MH in comparison to PH when shifting deadlines in cases 2M38, 2M46, and 6M163.

Case	Shift (days)	PH				MH				Gap in Obj.	
		Obj.		C.T.		Obj.		C.T.		Time unit (hours)	%Gap
		μ	σ	μ	σ	μ	σ	μ	σ		
2M38	−2	22.41	2.03	0.01	0.01	10.80	0.92	163.71	34.17	−11.61	−51.81
	−1	17.70	2.00	0.01	0.01	10.00	0.47	132.82	37.50	−7.70	−43.50
	0	16.70	1.42	0.01	0.00	10.10	0.32	101.68	25.61	−6.60	−39.52
	+1	17.90	2.23	0.01	0.00	10.30	0.48	124.20	36.79	−7.60	−42.46
	+2	17.30	1.57	0.01	0.00	10.40	0.52	102.02	26.80	−6.90	−39.88
2M46	−2	40.87	2.78	0.08	0.08	20.07	1.75	452.07	193.67	−20.80	−50.90
	−1	28.40	1.26	0.02	0.01	17.00	0.94	280.53	98.62	−11.40	−40.14
	0	28.20	1.99	0.02	0.01	17.10	0.32	272.03	54.38	−11.10	−39.36
	+1	27.40	1.51	0.01	0.00	17.30	1.16	251.68	98.42	−10.10	−36.86
	+2	27.60	1.84	0.01	0.00	17.00	0.47	280.53	55.73	−10.60	−38.41
6M163	−14	307.19	11.04	0.23	0.19	110.80	7.11	3535.75	20.58	−196.39	−63.93
	−7	99.90	3.25	0.08	0.02	69.50	1.96	2771.41	973.49	−30.40	−30.43
	0	79.90	2.51	0.07	0.02	59.50	2.17	2941.53	866.83	−20.40	−25.53
	+7	89.60	2.50	0.07	0.01	62.90	2.02	3591.52	72.78	−26.70	−29.80
	+14	87.80	3.36	0.07	0.03	65.00	3.30	2956.90	819.73	−22.80	−25.97

References

Ahmadi, E., Goldengorin, B., Süer, G. A., & Mosadegh, H. (2018). A hybrid method of 2-TSP and novel learning-based GA for job sequencing and tool switching problem. *Applied Soft Computing, 65*, 214–229.

Al-Fawzan, M., & Al-Sultan, K. (2003). A tabu search based algorithm for minimizing the number of tool switches on a flexible machine. *Computers & Industrial Engineering, 44*(1), 35–47.

Amaya, J. E., Cotta, C., & Fernández, A. J. (2008). A memetic algorithm for the tool switching problem. In M. J. Blesa, C. Blum, C. Cotta, A. J. Fernández, J. E. Gallardo, A. Roli, & M. Sampels (Eds.), *Hybrid metaheuristics*. In *Lecture Notes in Computer Science: 5296* (pp. 190–202). Berlin, Heidelberg: Springer Berlin Heidelberg.

Amaya, J. E., Cotta, C., & Fernández-Leiva, A. J. (2011). Memetic cooperative models for the tool switching problem. *Memetic Computing, 3*(3), 199–216.

Amaya, J. E., Cotta, C., & Fernandez-Leiva, A. J. (2012). Solving the tool switching problem with memetic algorithms. *Artificial Intelligence for Engineering Design, Analysis and Manufacturing, 26*(2), 221–235.

Amaya, J. E., Cotta, C., & Leiva, A. J. F. (2010). A memetic cooperative optimization schema and its application to the tool switching problem. In R. Schaefer, C. Cotta, J. Kołodziej, & G. Rudolph (Eds.), *Parallel problem solving from nature, ppsn xi*. In *Lecture Notes in Computer Science: 6238* (pp. 445–454). Berlin, Heidelberg: Springer Berlin Heidelberg.

Baykasoğlu, A., & Ozsoydan, F. B. (2017). Minimizing tool switching and indexing times with tool duplications in automatic machines. *The International Journal of Advanced Manufacturing Technology, 89*(5), 1775–1789.

Baykasoğlu, A., & Ozsoydan, F. B. (2018). Minimisation of non-machining times in operating automatic tool changers of machine tools under dynamic operating conditions. *International Journal of Production Research, 56*(4), 1548–1564.

Beezão, A. C., Cordeau, J.-F., Laporte, G., & Yanasse, H. H. (2017). Scheduling identical parallel machines with tooling constraints. *European Journal of Operational Research, 257*(3), 834–844.

Box, G., Hunter, J., & Hunter, W. (2005). *Statistics for experimenters: Design, innovation, and discovery*. Wiley-Interscience.

Burger, A. P., Jacobs, C. G., van Vuuren, J. H., & Visagie, S. E. (2015). Scheduling multi-colour print jobs with sequence-dependent setup times. *Journal of Scheduling, 18*(2), 131–145.

Calmels, D. (2019). The job sequencing and tool switching problem: state-of-the-art literature review, classification, and trends. *International Journal of Production Research, 57*(15–16), 5005–5025.

Catanzaro, D., Gouveia, L., & é, M. (2015). Improved integer linear programming formulations for the job Sequencing and tool Switching Problem. *European Journal of Operational Research, 244*(3), 766–777.

Chaves, A., Lorena, L., Senne, E., & Resende, M. (2016). Hybrid method with CS and BRKGA applied to the minimization of tool switches problem. *Computers & Operations Research, 67*, 174–183.

Crama, Y., Kolen, A. W. J., Oerlemans, A. G., & Spieksma, F. C. R. (1994). Minimizing the number of tool switches on a flexible machine. *International Journal of Flexible Manufacturing Systems, 6*(1), 33–54.

Dang, Q.-V., Nguyen, C. T., & Rudová, H. (2019). Scheduling of mobile robots for transportation and manufacturing tasks. *Journal of Heuristics, 25*(2), 175–213.

Djellab, H., Djellab, K., & Gourgand, M. (2000). A new heuristic based on a hypergraph representation for the tool switching problem. *International Journal of Production Economics, 64*(1), 165–176.

Du, K.-L., & Swamy, M. N. S. (2016). *Search and optimization by metaheuristics: Techniques and algorithms inspired by nature*. Springer, Switzerland.

Fathi, Y., & Barnette, K. W. (2002). Heuristic procedures for the parallel machine problem with tool switches. *International Journal of Production Research, 40*(1), 151–164.

Furrer, M., & Mütze, T. (2017). An algorithmic framework for tool switching problems with multiple objectives. *European Journal of Operational Research, 259*(3), 1003–1016.

Gao, J., Sun, L., & Gen, M. (2008). A hybrid genetic and variable neighborhood descent algorithm for flexible job shop scheduling problems. *Computers & Operations Research, 35*(9), 2892–2907.

Gen, M., Cheng, R., & Lin, L. (2008). *Network models and optimization: Multiobjective genetic algorithm approach*. Springer, London.

Ghiani, G., Grieco, A., & Guerriero, E. (2010). Solving the job sequencing and tool switching problem as a nonlinear least cost Hamiltonian cycle problem. *Networks, 55*(4), 379–385.

Ghrayeb, O. A., Phojanamongkolkij, N., & Finch, P. R. (2003). A mathematical model and heuristic procedure to schedule printed circuit packs on sequencers. *International Journal of Production Research, 41*(16), 3849–3860.

Goldberg, D. E., & Lingle, R., Jr. (1985). Alleles, loci, and the traveling salesman problem. In J. J. Grefenstette (Ed.), *Proceedings of the first international conference on genetic algorithms* (pp. 154–159). Hillsdale, NJ, USA: L. Erlbaum Associates Inc..

Guimarães, L., Klabjan, D., & Almada-Lobo, B. (2013). Pricing, relaxing and fixing under lot sizing and scheduling. *European Journal of Operational Research, 230*(2), 399–411.

Gökgür, B., Hnich, B., & Özpeynirci, S. (2018). Parallel machine scheduling with tool loading: A constraint programming approach. *International Journal of Production Research, 56*(16), 5541–5557.

Hertz, A., Laporte, G., Mittaz, M., & Stecke, K. E. (1998). Heuristics for minimizing tool switches when scheduling part types on a flexible machine. *IIE Transactions, 30*(8), 689–694.

Jans, R. (2009). Solving lot-sizing problems on parallel identical machines using symmetry-breaking constraints. *INFORMS Journal on Computing, 21*(1), 123–136.

Karakayalı, İ., & Azizoğlu, M. (2006). Minimizing total flow time on a single flexible machine. *International Journal of Flexible Manufacturing Systems, 18*(1), 55–73.

Keung, K. W., Ip, W. H., & Lee, T. C. (2001a). A genetic algorithm approach to the multiple machine tool selection problem. *Journal of Intelligent Manufacturing, 12*(4), 331–342.

Keung, K. W., Ip, W. H., & Lee, T. C. (2001b). The solution of a multi-objective tool selection model using the GA approach. *The International Journal of Advanced Manufacturing Technology, 18*(11), 771–777.

Khan, B. K., Gupta, B. D., Gupta, D. K. S., & Kumar, K. D. (2000). A generalized procedure for minimizing tool changeovers of two parallel and identical CNC machining centres. *Production Planning & Control, 11*(1), 62–72.

Laporte, G., Salazar-González, J. J., & Semet, F. (2004). Exact algorithms for the job sequencing and tool switching problem. *IIE Transactions, 36*(1), 37–45.

Lin, L., Shinn, S. W., Gen, M., & Hwang, H. (2006). Network model and effective evolutionary approach for AGV dispatching in manufacturing system. *Journal of Intelligent Manufacturing, 17*(4), 465–477.

Lin, S.-W., & Ying, K.-C. (2016). Optimization of makespan for no-wait flowshop scheduling problems using efficient matheuristics. *Omega, 64*, 115–125.

Paiva, G. S., & Carvalho, M. A. M. (2017). Improved heuristic algorithms for the job sequencing and tool switching problem. *Computers & Operations Research, 88*, 208–219.

Rabbani, M., Montazeri, M., Farrokhi-Asl, H., & Rafiei, H. (2016). A multi-objective genetic algorithm for a mixed-model assembly U-line balancing type-I problem considering human-related issues, training, and learning. *Journal of Industrial Engineering International, 12*(4), 485–497.

Raduly-Baka, C., Knuutila, T., & Nevalainen, O. S. (2005). Minimising the Number of Tool Switches with Tools of Different Sizes. *TUCS Technical Reports*. Turku Centre for Computer Science.

Reeves, C. R. (2010). Genetic Algorithms. In M. Gendreau, & J.-Y. Potvin (Eds.), *Handbook of metaheuristics*. In *International Series in Operations Research & Management Science* (pp. 109–139). Springer US.

Salonen, K., Raduly-Baka, C., & Nevalainen, O. S. (2006). A note on the tool switching problem of a flexible machine. *Computers & Industrial Engineering, 50*(4), 458–465.

Sarmadi, H., & Gholami, S. (2011). Modeling of tool switching problem in a flexible manufacturing cell: with two or more machines. In Y. Xie (Ed.), *Proceeding of the international conference on mechanical and electrical technology, 3rd, (ICMET-China 2011), Volumes 1–3* (pp. 2345–2349). ASME Press.

Schwerdfeger, S., & Boysen, N. (2017). Order picking along a crane-supplied pick face: The SKU switching problem. *European Journal of Operational Research, 260*(2), 534–545.

Sherali, H. D., & Smith, J. C. (2001). Improving discrete model representations via symmetry considerations. *Management Science, 47*(10), 1396–1407.

Shivanand, H. K., Benal, M. M., & Koti, V. (2006). *Flexible manufacturing system*. New Age International.

Solimanpur, M., & Rastgordani, R. (2012). Minimising tool switching and indexing times by ant colony optimisation in automatic machining centres. *International Journal of Operational Research, 13*(4), 465–479.

Tang, C. S., & Denardo, E. V. (1988). Models arising from a flexible manufacturing machine, part I: minimization of the number of tool switches. *Operations Research, 36*(5), 767–777.

Tzur, M., & Altman, A. (2004). Minimization of tool switches for a flexible manufacturing machine with slot assignment of different tool sizes. *IIE Transactions, 36*(2), 95–110.

Unlu, Y., & Mason, S. J. (2010). Evaluation of mixed integer programming formulations for non-preemptive parallel machine scheduling problems. *Computers & Industrial Engineering, 58*(4), 785–800.

Van Hop, N. (2005). The tool-switching problem with magazine capacity and tool size constraints. *IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans, 35*(5), 617–628.

Van Hop, N., & Nagarur, N. N. (2004). The scheduling problem of pcbs for multiple non-identical parallel machines. *European Journal of Operational Research, 158*(3), 577–594.

Weng, M. X., & Ren, H. (2006). An efficient priority rule for scheduling job shops to minimize mean tardiness. *IIE Transactions, 38*(9), 789–795.

Woo, Y.-B., & Kim, B. S. (2018). Matheuristic approaches for parallel machine scheduling problem with time-dependent deterioration and multiple rate-modifying activities. *Computers & Operations Research, 95*, 97–112.

Özpeynirci, S., Gökgür, B., & Hnich, B. (2016). Parallel machine scheduling with tool loading. *Applied Mathematical Modelling, 40*(9–10), 5660–5671.